nox-mem: Pain-Weighted Hybrid Memory for LLM Agents

Luiz Antonio Busnello *Independent Researcher*

Version v1.0.1 (2026-09-04) — changelog in `paper/CHANGELOG.md`

The system described here runs on a single 4-vCPU / 8 GB virtual private server under Debian Linux, serving six agents over a private overlay network. Concrete hardware and hosting details are immaterial to every result reported below and are omitted; §5.7 states the operational envelope in machine-independent terms (latency percentiles, resident set size, cost per query).

Abstract

We introduce **nox-mem**, a persistent memory system for autonomous LLM agents built on one principle: **pain-weighted hybrid memory with shadow discipline**. Retrieval and retention are governed by an additive salience formula in which *pain* — an operator-assigned severity in [0.1, 1.0], persisted on every chunk — is a first-class signal, and ranking changes pass a mandatory shadow phase before production activation. The system is a single SQLite file with provider-swappable embeddings, MIT-licensed: no vendor lock-in, sub-second writeback (inotifywait-driven), per-`chunk_type` retention windows, chunk-level provenance, and a policy-gated pre-snapshot before destructive operations. Deployed in production since March 14, 2026, it serves six specialized agents at KG-path p50 = 2.9 ms, $0 per KG-path query, and a 399 MB resident set in a single self-hosted process (§5.7). Our central result is a pre-registered, same-corpus comparison against five competing memory systems, of which four produced head-to-head quality numbers — two in the 2026-06-15 canonical run and two (EverOS and Zep) in later runs over the same corpus and query set — and one was a documented deployment non-run (§6). Under each system's native embedder the two leaders **split** — Mem0 wins LoCoMo (nDCG@10 0.469 vs 0.426), nox-mem wins LongMemEval. An embedding-matched variant (both Gemini 3072-d, full n = 2,482) — planned rather than post-hoc (§6.7), and an embedding match rather than a clean architecture isolation (§6.3.2) — **inverts the split**: nox-mem leads on both datasets (LongMemEval 0.526 vs 0.406; LoCoMo 0.495 vs 0.441) and in all five represented categories, with four residual confounds declared. On EverMemBench, nox-mem reaches **63.28% Overall** with Gemini-3-flash, above every MemOS Table 4 number — all of which were obtained on GPT-4.1-mini, so the backbones differ and this is not a state-of-the-art claim (§5.1.10). Three findings cut against our own headline: *pain*'s isolated retrieval effect is directional but not statistically significant (§7.1); section-aware ranking, not pain, is the dominant empirical driver (§5.1.3); and on the same EverMemBench run the F_MH multi-hop track sits at 3–7%, against 18.88% strict EM for the best published system on that track, which §5.4 attributes

principally to task setup.

---

1. Introduction

1.1 Problem Statement

Large Language Model (LLM) agents operating in production environments face a fundamental limitation: context window ephemerality. When a conversation ends or context is compacted, the agent loses accumulated knowledge, decisions, and operational state. For multi-agent systems where specialized agents collaborate on complex tasks, this problem compounds — agents cannot learn from each other's experiences, cannot recall past decisions, and cannot build institutional knowledge over time.

1.2 Design Goals

nox-mem was designed with four core objectives:

1. **Persistent Memory**: Survive context window resets, session boundaries, and agent restarts
2. **Intelligent Retrieval**: Return semantically relevant results, not just keyword matches
3. **Cross-Agent Intelligence**: Enable knowledge sharing across isolated agent workspaces
4. **Operational Autonomy**: Self-maintain through automated consolidation, pruning, and indexing

1.3 Scope

The system operates within the OpenClaw platform, serving 6 AI agents (Nox, Atlas, Boris, Cipher, Forge, Lex) on a single VPS with 4 vCPUs and 8GB RAM. Each agent has a distinct role and memory profile. The workspace (shared memory) and individual agent databases form a federated memory architecture.

1.4 Related Memory Systems and the Six Gaps

The published memory-for-LLM-agents literature spans roughly three families: (i) *vector-store wrappers with metadata layers* — **mem0** [1], **Letta** [2]; (ii)

[1] Chhikara, Khant, Aryan, Singh & Yadav, *Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory*, arXiv:2504.19413, 2025. Implementation: `mem0ai/mem0` — open-source memory layer for LLM agents (PostgreSQL + Qdrant backend, OpenAI embeddings by default), github.com/mem0ai/mem0. Used in §1.4, §6.3, Table 2.

[2] Letta (formerly MemGPT) — agent-loop memory architecture with archival/recall memory separation. Original system paper: Packer et al., *MemGPT: Towards LLMs as Operating Systems*, arXiv:2310.08560. github.com/letta-ai/letta. Used in §1.4, §6.3, Table 2.

temporal- and provenance-aware memory services — **Zep** [3], **memanto**; (iii) *KG-augmented and graph-fused retrieval* — **LightRAG** [4] (HKU, EMNLP 2025), **HippoRAG2** [5]; and (iv) *parametric-memory paradigms*, most recently **MeMo** [6] (which folds reflections into model weights via continued pretraining — the design opposite of ours). **EverMind-AI/EverOS** [7] occupies a distinct slot: it is the only memory OS in this space that publishes its own benchmark dataset (EverMemBench) and reports threshold numbers, raising the bar for honest cross-system comparison (§6). Four works outside these families bound the problem this design answers. *Lost in the Middle*[8] shows that simply extending the context window degrades mid-context recall, which is why retrieval — not a larger prompt — is the mechanism here. **Reflexion**[9] established verbal self-reflection as an agent-improvement loop, the precursor of the reflection artefacts our `crystallize` path persists (§2.5). **A-Mem**[10] pursues agent-managed memory organisation, an axis orthogonal to our retrieval-side

[3]Rasmussen, Paliychuk, Beauvais, Ryan & Chalef, *Zep: A Temporal Knowledge Graph Architecture for Agent Memory*, arXiv:2501.13956, 2025. Implementation: github.com/getzep/zep. Requires one paid LLM key to boot (OpenAI by default, Anthropic selectable); embeddings may run keyless against its own local embedder service. Verified in the v0.27.2 source — see [^zep-stack]. Used in §1.4, §6.3, Table 2.

[4]Guo et al., *LightRAG: Simple and Fast Retrieval-Augmented Generation*, EMNLP 2025 (HKU); arXiv:2410.05779. github.com/HKUDS/LightRAG (~35k stars, MIT). Cited in §1.4 as a KG-augmented baseline; §3.4 references its LLM-summarized incremental KG-merge pattern as a forward-looking optimization (LightRAG-style summarization deferred until KG density reaches ~10× the current value).

[5]Gutiérrez, Shu, Qi, Zhou & Su, *From RAG to Memory: Non-Parametric Continual Learning for Large Language Models* (HippoRAG 2), ICML 2025. arXiv:2502.14802. Graph-augmented retrieval with Personalized PageRank over an entity-relation graph; cited as a graph-baseline peer in §1.4, §1.5 and §6. **Caveat.** This footnote carried **no locator at all** until 2026-09-09 — an irresolvable reference is indistinguishable from an invented one, which is why `footnotes_check` now requires one. **Caveat.** And requiring a locator is not sufficient: until 2026-09-10 this same footnote named two authors who are not on that paper, and the check passed because it verifies the *presence* of an identifier and not its *correspondence* to the names beside it. Three of the manuscript's 32 arXiv-bearing footnotes carried wrong author surnames under a green check.

[6]arXiv 2605.15156v2, *MeMo: Towards Language Models with Associative Memory Mechanisms* (parametric reflections folded into model weights). Cited as the design opposite of nox-mem's externalized, inspectable memory paradigm. §1.4, abstract.

[7]Hu, Gao, Zhou, Xu, Bai, Li, Zhang, Li, Zhang, Bing & Deng, *EverMemOS: A Self-Organizing Memory Operating System for Structured Long-Horizon Reasoning*, arXiv:2601.02163, 2026. Implementation: github.com/EverMind-AI (~5k stars, Apache 2.0). Publishes EverMemBench dataset and an EvoAgentBench-framed evolution loop. The only memory-OS peer in our taxonomy that publishes its own benchmark; §3.4 is the direct narrative counter to EvoAgent framing. Honest-comparison action item: run nox-mem on EverMemBench (queued as F4 in §7.2). This is also the system paper for what §6.3.3 measures — §6.3.3 reports a quality number **of** this system, and citing only its repository would leave the reader unable to check what was built. Its reported 93.05% LoCoMo / 83.00% LongMemEval are LLM-judged answer accuracy, a different quantity from the retrieval nDCG@10 of §6.3.3, and are not compared to it.

[8]Liu, Lin, Hewitt, Paranjape, Bevilacqua, Petroni & Liang, *Lost in the Middle: How Language Models Use Long Contexts*, TACL 2024. arXiv:2307.03172. Used in §1.4 as the bound on context-window scaling.

[9]Shinn, Cassano, Berman, Gopinath, Narasimhan & Yao, *Reflexion: Language Agents with Verbal Reinforcement Learning*, NeurIPS 2023 (vol. 36). arXiv:2303.11366. Used in §1.4.

[10]Xu et al., *A-Mem: Agentic Memory for LLM Agents*, 2025. arXiv:2502.12110. Used in §1.4.

contribution. And **HaluMem**[11] measures hallucination *in memory systems specifically* — an evaluation dimension we do **not** report, and therefore a declared gap rather than a claimed strength (§7.2).

Across these systems, six recurring gaps appear in the design space, and each motivates a concrete subsystem below. The coverage table is compiled from each system's own documentation rather than from measurement.

Table 1 — Design-space coverage of the six gaps, as reported by each system's own documentation.

What this table is and is not. Only the `nox-mem` column is measured here. Every other cell records what that system's paper, README, or API reference *states* about itself, read between 2026-05 and 2026-06; we did not deploy the other six systems to verify their claims, and §6 explains why only two of them (Mem0 and agentmemory) could be brought to a same-corpus comparison at all. A cell therefore answers "does the system claim this property?", not "does the system have it?". `n/r` means the source material does not address the gap either way — it is an absence of documentation, not a negative finding. Read the table as a map of the design space that motivates §§2–4, and not as an evaluation of competing systems.

#	Gap	nox-mem (measured)	mem0	Letta	Zep	EverOS	LightRAG	MeMo
1	Static injection	yes — live writeback	partial	yes	yes	yes	no — batch	no — retrain
2	No temporal decay	yes — salience + retention	no	no	yes	partial	no	no
3	No provenance	yes — chunk_id + source_file	partial	yes	yes	yes	yes	no — baked-in
4	Flat memory	yes — KG + section_boost	no	no	yes	yes — hypergraph	yes — dual-level	no
5	No write-back	yes — crystallize + reflect + consolidate	partial	yes	yes	yes — EvoAgent	no	no
6	Indexing delay	yes — inotifywait < 1 s	`n/r`	yes	yes	`n/r`	partial — batch	no — retrain

[11] *HaluMem: Evaluating Hallucinations in Memory Systems of Agents*, 2025. arXiv:2511.03506. Used in §1.4 as a declared evaluation gap.

Sources for the non-`nox-mem` columns: mem0 [12], Letta [13], Zep [14], EverOS [15], LightRAG [16], MeMo [17].

How each gap is closed. Live writeback addresses static injection (§3.1); typed `retention_days` folded into the salience formula addresses the absence of temporal decay (§3.4.3); `chunk_id` and `source_file` on every result, plus an append-only `ops_audit` table behind `withOpAudit()`, address provenance (§3.3); FTS5, typed retention, an extracted knowledge graph and section-aware `section_boost` address flat memory (§4.1); the `crystallize/reflect/consolidate` triad addresses the absence of writeback (§3.4); and sub-second re-ingestion addresses indexing delay (§3.1).

The single design principle that ties these closures together is **pain weighting under shadow discipline**: every chunk carries an explicit `pain` severity, every ranking change rolls out first in `NOX_SALIENCE_MODE=shadow` with /api/health telemetry [18], and only graduates to `active` after operator review.

---

[12]Chhikara, Khant, Aryan, Singh & Yadav, *Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory*, arXiv:2504.19413, 2025. Implementation: `mem0ai/mem0` — open-source memory layer for LLM agents (PostgreSQL + Qdrant backend, OpenAI embeddings by default), github.com/mem0ai/mem0. Used in §1.4, §6.3, Table 2.

[13]Letta (formerly MemGPT) — agent-loop memory architecture with archival/recall memory separation. Original system paper: Packer et al., *MemGPT: Towards LLMs as Operating Systems*, arXiv:2310.08560. github.com/letta-ai/letta. Used in §1.4, §6.3, Table 2.

[14]Rasmussen, Paliychuk, Beauvais, Ryan & Chalef, *Zep: A Temporal Knowledge Graph Architecture for Agent Memory*, arXiv:2501.13956, 2025. Implementation: github.com/getzep/zep. Requires one paid LLM key to boot (OpenAI by default, Anthropic selectable); embeddings may run keyless against its own local embedder service. Verified in the v0.27.2 source — see [^zep-stack]. Used in §1.4, §6.3, Table 2.

[15]Hu, Gao, Zhou, Xu, Bai, Li, Zhang, Li, Zhang, Bing & Deng, *EverMemOS: A Self-Organizing Memory Operating System for Structured Long-Horizon Reasoning*, arXiv:2601.02163, 2026. Implementation: github.com/EverMind-AI (~5k stars, Apache 2.0). Publishes EverMemBench dataset and an EvoAgentBench-framed evolution loop. The only memory-OS peer in our taxonomy that publishes its own benchmark; §3.4 is the direct narrative counter to EvoAgent framing. Honest-comparison action item: run nox-mem on EverMemBench (queued as F4 in §7.2). This is also the system paper for what §6.3.3 measures — §6.3.3 reports a quality number **of** this system, and citing only its repository would leave the reader unable to check what was built. Its reported 93.05% LoCoMo / 83.00% LongMemEval are LLM-judged answer accuracy, a different quantity from the retrieval nDCG@10 of §6.3.3, and are not compared to it.

[16]Guo et al., *LightRAG: Simple and Fast Retrieval-Augmented Generation*, EMNLP 2025 (HKU); arXiv:2410.05779. github.com/HKUDS/LightRAG (~35k stars, MIT). Cited in §1.4 as a KG-augmented baseline; §3.4 references its LLM-summarized incremental KG-merge pattern as a forward-looking optimization (LightRAG-style summarization deferred until KG density reaches ~10× the current value).

[17]arXiv 2605.15156v2, *MeMo: Towards Language Models with Associative Memory Mechanisms* (parametric reflections folded into model weights). Cited as the design opposite of nox-mem's externalized, inspectable memory paradigm. §1.4, abstract.

[18]`NOX_SALIENCE_MODE` environment variable controls the three-state gate (`shadow` | `active` | `off`). Default is `shadow`. Telemetry exposed at `/api/health.salience`. See §3.4.3 and `staged/1.7a/edits/salience.ts`.

1.5 Related Work

§1.4 compares nox-mem against deployable memory *products*. This section situates it in the *literature*, and its purpose is as much to mark what is **not** novel here as to locate the contribution.

Where this sits on the map. Two surveys chart the area this work belongs to. Zhang et al.[19] taxonomise agent memory by *what* is stored and *how* it is written, read and managed; Gao et al.[20] survey self-evolving agents along *what, when, how and where* to evolve. Read against either taxonomy, nox-mem is a narrow instance: a single-substrate store (SQLite, one embedding model) with a **hand-specified** write and retention policy, no learned component anywhere in the loop, and self-evolution limited to two operator-invoked primitives (§3.4). Most of the axes those surveys enumerate are, for this system, fixed at their simplest setting. That is the point of comparison worth making — the results in §5–§6 come from a system that declines nearly every degree of freedom the literature has opened.

Retrieval substrate: deliberately conventional. Layer 1 is FTS5 BM25[21], Layer 2 is dense retrieval over a single embedding model, and the two are combined by Reciprocal Rank Fusion[22] at the standard `k=60`. Each of those choices has a canonical source and a stronger alternative we did not adopt. Dense retrieval for open-domain QA was established by DPR[23] and extended to unsupervised training by Contriever[24]; late-interaction scoring as in ColBERT[25] is more expressive than the single-vector cosine we use; generative fusion over retrieved passages as in FiD[26] and the original RAG formulation[27] moves work

[19]Zhang, Bo, Ma, Li, Chen *et al.*, *A Survey on the Memory Mechanism of Large Language Model based Agents*, ACM TOIS 2025. arXiv:2404.13501. Used in §1.5.

[20]Gao, Geng, Hua, Hu, Juan *et al.*, *A Survey of Self-Evolving Agents: What, When, How, and Where to Evolve on the Path to Artificial Super Intelligence*, TMLR 2026. arXiv:2507.21046. Used in §1.5.

[21]Robertson & Zaragoza, *The Probabilistic Relevance Framework: BM25 and Beyond*, Foundations and Trends in Information Retrieval 3(4), 2009. doi:10.1561/1500000019. Cited in §3.1 for the BM25 ranking used by Layer 1 (FTS5).

[22]Cormack, Clarke & Buettcher, *Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods*, SIGIR 2009. doi:10.1145/1571941.1572114. Cited in §3.2 — this is the source of the `k=60` constant used by the fusion layer.

[23]Karpukhin, Oğuz, Min, Lewis, Wu, Edunov, Chen & Yih, *Dense Passage Retrieval for Open-Domain Question Answering*, EMNLP 2020. arXiv:2004.04906. The canonical source for the dense-retrieval half of Layer 2 (§1.5).

[24]Izacard, Caron, Hosseini, Riedel, Bojanowski, Joulin & Grave, *Unsupervised Dense Information Retrieval with Contrastive Learning*, 2021. arXiv:2112.09118. Cited in §1.5 for unsupervised dense-retriever training.

[25]Khattab & Zaharia, *ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT*, SIGIR 2020. arXiv:2004.12832. Cited in §1.5 as the more expressive alternative to the single-vector scoring used here.

[26]Izacard & Grave, *Leveraging Passage Retrieval with Generative Models for Open Domain Question Answering*, EACL 2021. arXiv:2007.01282. Cited in §1.5 as generative fusion in the reader, which nox-mem does not perform.

[27]Lewis, Perez, Piktus, Petroni, Karpukhin, Goyal, Küttler, Lewis, Yih, Rocktäschel, Riedel & Kiela, *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*, NeurIPS 2020.

into the reader, which nox-mem does not do at all — it returns ranked chunks and leaves generation to the calling agent. On the index side we run **exact** search via sqlite-vec[28] rather than an approximate structure such as HNSW[29]; §7.1 records that this is the binding constraint past ~100k vectors, and it is a scaling limitation, not a design claim. One assumption underneath all of it is untested here: nox-mem retrieves on **every** query. Mallen et al.[30] show that for sufficiently popular facts a model's parametric knowledge beats retrieval, and that retrieving anyway can make the answer worse — which makes always-retrieve a policy choice this paper never evaluates against the alternative. **The contribution of this paper is not the retriever.** Nothing in Layers 1–2 would surprise an IR reader, and that is intentional: it isolates what does change, which is the retention and ranking policy above them.

Memory scoring: the closest prior art, and the delta. Generative Agents[31] scores a memory stream by recency, importance and relevance, and retrieves the top-scoring items into a limited context — structurally the same shape as the additive salience formula of §3.4. We do not claim the shape as novel. Two things differ. First, nox-mem adds **pain**, an operator-assigned severity in [0.1, 1.0] *persisted on the chunk* rather than inferred by the model at write time; the operator, not the LLM, decides what hurt. Second, the score is not consumed by a paging loop but by a ranking layer whose changes must pass a shadow phase before activation (§3.4.3) — the policy is auditable and reversible in a way an in-prompt scoring heuristic is not.

Learning the memory policy instead of fixing it. A recent line of work makes the memory policy itself the object of training. Mem-α[32] learns *construction* — what to write and how to structure it — by reinforcement learning; Memory-R1[33] learns the management operations (add, update, delete, retain) with outcome rewards; Memory as Action[34] treats context curation as an action

arXiv:2005.11401. Engaged in §1.5 as the formulation nox-mem does **not** adopt: it returns ranked chunks and leaves generation to the caller.

[28]Garcia, *sqlite-vec: A Vector Search Extension for SQLite*, 2024. github.com/asg017/sqlite-vec. Cited in §3.1 as the vector-table implementation; a software reference, so the repository is the identifier.

[29]Malkov & Yashunin, *Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs*, 2016. arXiv:1603.09320. Cited in §1.5 and §7.1 as the approximate index nox-mem does **not** use — the exact-search ceiling of §7.1 is a direct consequence.

[30]Mallen, Asai, Zhong, Das, Khashabi & Hajishirzi, *When Not to Trust Language Models: Investigating Effectiveness of Parametric and Non-Parametric Memories*, ACL 2023. arXiv:2212.10511. Used in §1.5.

[31]Park, O'Brien, Cai, Morris, Liang & Bernstein, *Generative Agents: Interactive Simulacra of Human Behavior*, UIST 2023. arXiv:2304.03442. The closest prior art to the salience formula of §3.4 (recency × importance × relevance over a memory stream); §1.5 states the delta explicitly.

[32]Wang, Takanobu, Liang, Mao, Hu *et al.*, *Mem-α: Learning Memory Construction via Reinforcement Learning*, 2025. arXiv:2509.25911. Used in §1.5.

[33]Yan, Yang, Huang, Nie, Ding *et al.*, *Memory-R1: Enhancing Large Language Model Agents to Manage and Utilize Memories via Reinforcement Learning*, 2025. arXiv:2508.19828. Used in §1.5.

[34]Zhang, Shu, Ma, Lin, Wu & Sang, *Memory as Action: Autonomous Context Curation for Long-Horizon Agentic Tasks*, 2025. arXiv:2510.12635. Used in §1.5.

in the agent's own action space; MEM1[35] trains memory and reasoning jointly so that the retained state stays constant in size across turns. Every one of them replaces a hand-written rule with a learned one, and reports gains from doing so.

nox-mem takes the opposite position, and the trade is worth stating plainly rather than defending. The salience weights of §3.4 are **constants chosen by hand**; `pain` is assigned by the operator; retention windows are integers per type. Nothing here adapts. What that buys is a policy an operator can read in one screen, diff in git, and revert — and a change process (the shadow gate of §3.4.3) that can hold a proposed ranking change against production traffic before it takes effect. What it costs is precisely what those four papers demonstrate: a fixed policy cannot discover, per corpus, what a trained one finds. This paper does not measure that gap, and the gap is real.

Self-evolution: the same two moves, without a training loop. ReasoningBank[36] distils reusable reasoning strategies out of an agent's own successes and failures and retrieves them on later tasks — structurally what `crystallize` (§3.4.1) does when it turns a solved incident into a procedure. Reflective Memory Management[37] splits the problem into *prospective* reflection (deciding at write time what will matter) and *retrospective* reflection (re-ranking retrieved evidence after the fact), which maps onto the write-side `pain` assignment and the read-side `reflect` (§3.4.2) respectively. In both cases the mechanism in this paper is the cheaper half: `crystallize` is **invoked by the operator** rather than triggered by a trained policy, and `reflect` is a synchronous call over already ranked chunks rather than a learned re-ranking step. WebCoach[38] closes the loop across *sessions*, feeding distilled advice from past episodes into a fresh agent that has no memory of them — the same cross-session role that §3.5's brief plays, except that its advice is produced by a trained component while the brief is a ranked assembly. The shapes are prior art; what is new here is neither shape but the accounting of what they cost at $0 of training.

Forgetting: also prior art, with a different granularity. MemoryBank[39] implements decay on an Ebbinghaus-style curve, a single forgetting function applied uniformly. nox-mem instead uses **typed retention windows** per `chunk_type` (§2.3), with `NULL` meaning never-decay for the `feedback` and `person` types. The trade is expressiveness for inspectability: a per-type integer in a

[35]Zhou, Qu, Wu, Kim, Prakash *et al.*, *MEM1: Learning to Synergize Memory and Reasoning for Efficient Long-Horizon Agents*, 2025. arXiv:2506.15841. Used in §1.5.

[36]Ouyang, Yan, Hsu, Chen, Jiang *et al.*, *ReasoningBank: Scaling Agent Self-Evolving with Reasoning Memory*, ICLR 2026. arXiv:2509.25140. Used in §1.5 and §3.4.

[37]Tan, Yan, Hsu, Han, Wang *et al.*, *In Prospect and Retrospect: Reflective Memory Management for Long-term Personalized Dialogue Agents*, ACL 2025. arXiv:2503.08026. Used in §1.5 and §3.4.

[38]Liu, Geng, Li, Cui, Zhang *et al.*, *WebCoach: Self-Evolving Web Agents with Cross-Session Memory Guidance*, 2025. arXiv:2511.12997. Used in §1.5.

[39]Zhong, Guo, Gao, Ye & Wang, *MemoryBank: Enhancing Large Language Models with Long-Term Memory*, 2023. arXiv:2305.10250. The closest prior art to typed retention (§2.3), via Ebbinghaus-curve decay; §1.5 states the granularity difference.

column is coarser than a curve, and it is also legible in `sqlite3` and changeable without re-deriving anything.

Structure of the store. Rezazadeh et al.[40] grow a *dynamic* tree of schemas over a conversation, deepening it as material accumulates. nox-mem's entity files (§2.3) are the degenerate case of that idea: a **fixed** three-section shape — frontmatter, compiled truth, timeline — with a per-section retrieval boost and no restructuring at all. On the multi-agent side, MIRIX[41] coordinates several specialised memory components behind one interface; the architecture of §2.4 instead gives each of six agents its own database and reads across them with a single fan-out query (`cross_search`), which keeps per-agent provenance at the cost of any shared consolidation between them.

Agent-memory architectures. MemGPT[42] frames memory as an operating-system problem, paging between a fixed context window and archival storage. nox-mem does not page: the context window is populated by a brief (§3.5) assembled from a ranked query, and there is no eviction loop. The distinction matters for failure modes — a paging architecture can lose an item by evicting it, while a ranking architecture loses it by ranking it low, which is recoverable by changing the ranker and is exactly what §3.4.3's shadow gate exists to control. MemAgent[43] attacks the same context limit from a third direction, training an agent to rewrite a fixed-size memory across successive chunks of a long input; that is a *reading* strategy for one long document, where nox-mem's brief is an *assembly* strategy over a persistent multi-source store. A fourth direction compresses instead of selecting: ACON[44] optimises *how* long-horizon context is condensed, trading fidelity for room. The brief does neither — it ranks and truncates at K, keeping every retained chunk verbatim. That is what makes a brief auditable (every line in it exists in the store, unaltered) and it is also why it cannot fit more evidence into the same token budget than selection allows. Whether compression would beat selection at equal budget is untested here.

Graph-augmented retrieval. HippoRAG[45] applies Personalized PageRank over an entity-relation graph, and HippoRAG 2[46] extends the approach toward

[40]Rezazadeh, Li, Wei & Bao, *From Isolated Conversations to Hierarchical Schemas: Dynamic Tree Memory Representation for LLMs*, 2024. arXiv:2410.14052. Used in §1.5 and §2.3.

[41]Wang & Chen, *MIRIX: Multi-Agent Memory System for LLM-Based Agents*, 2025. arXiv:2507.07957. Used in §1.5 and §2.4.

[42]Letta (formerly MemGPT) — agent-loop memory architecture with archival/recall memory separation. Original system paper: Packer et al., *MemGPT: Towards LLMs as Operating Systems*, arXiv:2310.08560. github.com/letta-ai/letta. Used in §1.4, §6.3, Table 2.

[43]Yu, Chen, Feng, Chen, Dai *et al.*, *MemAgent: Reshaping Long-Context LLM with Multi-Conv RL-based Memory Agent*, ICLR 2026. arXiv:2507.02259. Used in §1.5.

[44]Kang, Chen, Han, Inan, Wutschitz *et al.*, *ACON: Optimizing Context Compression for Long-horizon LLM Agents*, 2025. arXiv:2510.00615. Used in §1.5.

[45]Gutiérrez, Shu, Gu, Yasunaga & Su, *HippoRAG: Neurobiologically Inspired Long-Term Memory for Large Language Models*, NeurIPS 2024. arXiv:2405.14831. Cited in §1.5 as the PPR-over-entity-graph approach the KG path deliberately does not implement.

[46]Gutiérrez, Shu, Qi, Zhou & Su, *From RAG to Memory: Non-Parametric Continual Learning for Large Language Models* (HippoRAG 2), ICML 2025. arXiv:2502.14802. Graph-augmented retrieval with Personalized PageRank over an entity-relation graph; cited as a graph-baseline peer in §1.4,

non-parametric continual learning; LightRAG[47] merges an incremental KG using LLM-generated summaries. nox-mem's KG path (§4.1) is far weaker by design: a SQL + regex entity walk over `kg_relations`, with **no PPR and no LLM call**, which is why it costs $0 and runs at single-digit milliseconds (§5.7). It buys latency and cost at the price of multi-hop expressiveness, and §5.4 quantifies that price rather than hiding it.

Benchmarks and evaluation. The long-term conversational memory setting is measured here on LoCoMo[48] and LongMemEval[49], both of which descend from the multi-session setup introduced by Beyond Goldfish Memory[50]; classical multi-hop QA on MuSiQue[51] against the IRCoT[52] baseline. Retrieval-quality methodology follows the zero-shot heterogeneous-benchmark discipline of BEIR[53] and the embedding-model evaluation conventions of MTEB[54], which is why §6 reports nDCG@10 under each system's native embedder *and* an embedding-matched variant — a single-embedder comparison conflates retriever quality with embedder quality. Multi-party long-horizon collaborative memory, evaluated by Hu et al.[55], is a setting nox-mem has **not** been measured on and

§1.5 and §6. **Caveat.** This footnote carried **no locator at all** until 2026-09-09 — an irresolvable reference is indistinguishable from an invented one, which is why `footnotes_check` now requires one. **Caveat.** And requiring a locator is not sufficient: until 2026-09-10 this same footnote named two authors who are not on that paper, and the check passed because it verifies the *presence* of an identifier and not its *correspondence* to the names beside it. Three of the manuscript's 32 arXiv-bearing footnotes carried wrong author surnames under a green check.

[47]Guo et al., *LightRAG: Simple and Fast Retrieval-Augmented Generation*, EMNLP 2025 (HKU); arXiv:2410.05779. github.com/HKUDS/LightRAG (~35k stars, MIT). Cited in §1.4 as a KG-augmented baseline; §3.4 references its LLM-summarized incremental KG-merge pattern as a forward-looking optimization (LightRAG-style summarization deferred until KG density reaches ~10× the current value).

[48]Maharana, Lee, Tulyakov, Bansal, Barbieri & Fang, *Evaluating Very Long-Term Conversational Memory of LLM Agents*, ACL 2024. aclanthology.org/2024.acl-long.747. The LoCoMo benchmark used in §5.5 and §6.

[49]Wu, Wang, Yu, Zhang, Chang & Yu, *LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory*, ICLR 2025. arXiv:2410.10813. The cross-bench validation set of §5.6.

[50]Xu, Szlam & Weston, *Beyond Goldfish Memory: Long-Term Open-Domain Conversation*, ACL 2022. arXiv:2107.07567. Used in §1.5 and §6.

[51]Trivedi, Balasubramanian, Khot & Sabharwal, *MuSiQue: Multihop Questions via Single-hop Question Composition*, TACL 2022. arXiv:2108.00573. The multi-hop QA dataset used in §5.2.

[52]Trivedi, Balasubramanian, Khot & Sabharwal, *Interleaving Retrieval with Chain-of-Thought Reasoning for Knowledge-Intensive Multi-Step Questions*, ACL 2023. arXiv:2212.10509. The IRCoT baseline compared against in §5.2.

[53]Thakur, Reimers, Rücklé, Srivastava & Gurevych, *BEIR: A Heterogeneous Benchmark for Zero-shot Evaluation of Information Retrieval Models*, NeurIPS 2021 (Datasets & Benchmarks). arXiv:2104.08663. The zero-shot evaluation discipline §6 follows.

[54]Muennighoff, Tazi, Magne & Reimers, *MTEB: Massive Text Embedding Benchmark*, EACL 2023. arXiv:2210.07316. The embedding-evaluation convention behind §6's native-embedder plus embedding-matched design.

[55]Hu et al., *Evaluating Long-Horizon Memory for Multi-Party Collaborative Agents* (the EverMemBench paper), 2026. arXiv:2602.01313. Named in §1.5 as a setting nox-mem has not been measured on — an open gap, not a claimed capability. §5.1.5–§5.1.10 report nox-mem numbers **on** this benchmark and §6.3.3 reports a number **of** the system whose authors also wrote it; both facts are disclosed rather than left implicit, and the disclosure applies symmetrically to our own use of it.

is named here as an open gap rather than a claimed capability.

Agent reasoning loops. The orchestration experiments of §5.5 are implementations of published loops, not new ones: IterB follows ReAct[56] and IterC follows the Self-Ask[57] decomposition. Their contribution in this paper is the measurement of where each loop's ceiling sits on top of this retriever, not the loops themselves. The newer alternative is to train the loop rather than script it: MemSearcher[58] learns jointly to reason, issue searches and prune its own context end-to-end. Untrained scripted loops are what §5.5 measures, and the ceilings it reports should be read as ceilings *of that class*.

2. System Architecture

2.1 Infrastructure Overview

The system runs on a Hostinger KVM4 VPS accessible via Tailscale VPN (a private Tailscale address). Five systemd-managed services provide the runtime environment:

Service	Port	Type	Function
openclaw-gateway	18789	WebSocket	Agent communication gateway
nox-mem-watcher	—	inotifywait	Filesystem event monitor
nox-mem-api	18800	HTTP/JSON	Dashboard data API
ollama	11434	HTTP	Local LLM inference (llama3.2:3b)
tailscaled	—	WireGuard	VPN mesh connectivity

2.2 Database Schema

The primary storage is SQLite 3 with WAL (Write-Ahead Logging) mode for concurrent access. The schema (version 3) contains:

Core Tables:

- `chunks` — Memory fragments with full-text indexing
  - `id` (INTEGER PK), `source_file` (TEXT), `chunk_text` (TEXT), `chunk_type` (TEXT)
  - `source_date` (TEXT), `is_consolidated` (INTEGER), `memory_type` (TEXT)
  - `created_at`, `updated_at` (TEXT, ISO 8601)
  - `metadata` (TEXT, JSON)
- `chunks_fts` — FTS5 virtual table with porter unicode61 tokenizer

[56]Yao, Zhao, Yu, Du, Shafran, Narasimhan & Cao, *ReAct: Synergizing Reasoning and Acting in Language Models*, ICLR 2023. arXiv:2210.03629. The loop implemented as IterB in §5.5.

[57]Press, Zhang, Min, Schmidt, Smith & Lewis, *Measuring and Narrowing the Compositionality Gap in Language Models*, 2022. arXiv:2210.03350. The decomposition implemented as IterC in §5.5.

[58]Yuan, Lou, Li, Chen, Lu *et al.*, *MemSearcher: Training LLMs to Reason, Search and Manage Memory via End-to-End Reinforcement Learning*, ACL 2026. arXiv:2511.02805. Used in §1.5.

  - Content-sync triggers (INSERT, UPDATE, DELETE) maintain index consistency
  - BM25 ranking[59] with configurable column weights (1.0, 0.5, 0.5)
- `consolidated_files` — Processing state tracker
  - `source_file` (TEXT PK), `status` (INTEGER: 0=pending, 1=done, -1=failed)
- `meta` — Key-value configuration store (schema_version, cursors, metrics)

Knowledge Graph Tables:

- `kg_entities` — Named entities with type classification and mention counting
  - UNIQUE constraint on (name, entity_type)
  - TTL tracking via `first_seen`, `last_seen`
- `kg_relations` — Typed relationships between entities
  - Confidence scoring (0.0-1.0) with temporal decay
  - TTL via `expires_at` (90-day default), `last_confirmed`
  - Evidence linking via `evidence_chunk_id`
- `decision_versions` — Architectural decision version history
  - Supersession chain via `is_current` flag and `superseded_at` timestamp

Vector Tables (sqlite-vec[60]):

- `vec_chunks` — Virtual table storing float32 embeddings (3072 dimensions)
- `vec_chunk_map` — Rowid-to-chunk_id mapping (sqlite-vec requires rowid-based access)
- `dedup_log` — Suppressed duplicate tracking for audit

2.3 Chunk Type Taxonomy

Memory chunks are classified into 10 types based on source file path patterns:

Type	Source Pattern	Current Count	Purpose
team	`shared/`	499	Shared team knowledge, cross-agent docs
daily	`memory/YYYY-MM-DD`	161	Daily operational notes
other	(default)	126	Unclassified content
decision	`memory/decisions.md`	34	Architectural and strategic decisions

[59]Robertson & Zaragoza, *The Probabilistic Relevance Framework: BM25 and Beyond*, Foundations and Trends in Information Retrieval 3(4), 2009. doi:10.1561/1500000019. Cited in §3.1 for the BM25 ranking used by Layer 1 (FTS5).

[60]Garcia, *sqlite-vec: A Vector Search Extension for SQLite*, 2024. github.com/asg017/sqlite-vec. Cited in §3.1 as the vector-table implementation; a software reference, so the repository is the identifier.

Type	Source Pattern	Current Count	Purpose
lesson	`memory/lessons.md`	21	Errors, corrections, learnings
project	`memory/projects.md`	11	Active project tracking
pending	`memory/pending.md`	8	Incomplete tasks and blockers
feedback	`memory/feedback/`	6	User and system feedback
person	`memory/people.md`	6	People profiles and contacts
digest	`memory/digests/`	2	Weekly summary reports

2.4 Multi-Agent Memory Architecture

Each of the 6 agents operates with an isolated database at `agents/{name}/tools/nox-mem/nox-mem.db`, relative to the workspace root. The `OPENCLAW_WORKSPACE` environment variable controls path resolution across all modules, enabling the same nox-mem binary to operate on different databases depending on the calling context.

Agent Memory Distribution (as of March 23, 2026):

Agent	Role	Chunks	DB Size	Dominant Type
Nox	Chief of Staff	185	268 KB	daily (91)
Boris	Head of Communications	148	268 KB	team (50)
Forge	Code Reviewer	182	292 KB	daily (135)
Atlas	Research	30	128 KB	other (17)
Cipher	Security	31	132 KB	other (12)
Lex	Legal/Compliance	31	132 KB	other (12)
Workspace	**Shared**	**874**	**25.2 MB**	**team (499)**

Total system memory: 1,481 chunks across 7 databases (initial-deployment snapshot, March 2026; the main store measured 67,724 chunks on 2026-09-09).

2.5 User-Facing Primitives

nox-mem exposes a deliberately small public contract: **three primitives**, all backed by the same SQLite store and surfaced identically across three transport layers (CLI, HTTP API, MCP). Every advanced verb (`reflect`, `cross-search`, `kg-path`, `crystallize`) decomposes internally into sequences of these three primitives.

Primitive 1 — `search` (hybrid retrieval). FTS5 BM25 + Gemini semantic (3072d) → Reciprocal Rank Fusion (RRF)[61], k=60, with optional Hard Mutex section gating and SOURCE_TYPE_BOOST overlays. Returns ranked chunks

[61] Cormack, Clarke & Buettcher, *Reciprocal Rank Fusion Outperforms Condorcet and Individual*

with `score`, `match_type`, and provenance fields (`source_file`, `section`, `created_at`, `updated_at`). Detailed in §4.

Primitive 2 — `answer` (grounded RAG). Internally calls `search` with `topK = 10`, builds a citation-anchored prompt over the retrieved chunks, invokes the configured LLM (`gemini-2.5-flash-lite` by default per D41), and parses inline `[chunk_<id>]` citations. Anti-hallucination guard: citations pointing to chunks outside the retrieved set trigger a single retry with a stricter prompt; a second failure raises `AnswerError('hallucination_after_retry')`. Empty-retrieval short-circuit avoids LLM spend when no chunks match. Measured p95 latency: 101.74 ms on the offline mock-LLM bench (42× under the 4.3 s budget); live p95 with Gemini Flash Lite ranges 1.5–2.5 s. Implementation: `staged/P1/edits/src/lib/answer/{index,retrieval,prompt,provider,config}.ts`.

Primitive 3 — Temporal filter (`--as-of` / `--changed-since`). Time-travel and recency-window selectors implemented as hard SQL pre-filters — not ranking boosts. `--as-of <date>` restricts to chunks satisfying `created_at <= date AND (deleted_at IS NULL OR deleted_at > date)`; `--changed-since <date>` restricts to chunks satisfying `updated_at > date OR created_at > date`. Combined, the two clauses AND. Accepted formats: ISO 8601 (`2026-05-01` or full `2026-05-01T00:00:00Z`) and relative (`7d`, `1w`, `30d`, `2h`, `15m`). Uses existing `chunks.created_at` and `chunks.updated_at` columns from schema v18 — no schema changes, no ranking changes. The filter is orthogonal to the E13 temporal proximity boost (`NOX_TEMPORAL_PATH`, §5), which additively reweights ranking by recency rather than restricting the candidate set. Implementation: `staged/P3/edits/{dates,search,api-server}.ts`.

Composition. The three primitives compose orthogonally — for example, `answer "what incidents happened last week?" --changed-since 7d` retrieves only chunks updated in the last seven days, then synthesizes a grounded answer over that restricted candidate set. This closure property — three small primitives, deterministic semantics, identical surface across transports — is the contract that makes nox-mem composable from agent runtimes that have no prior knowledge of the implementation.

---

3. Memory Pipeline

3.1 Ingestion

Files created or modified in monitored directories trigger the inotifywait-based watcher service[62]. The watcher implements:

- **Debounce logic**: 2-second delay to batch rapid successive writes
- **File filtering**: Only `.md` and `.json` files are processed

Rank Learning Methods, SIGIR 2009. doi:10.1145/1571941.1572114. Cited in §3.2 — this is the source of the `k=60` constant used by the fusion layer.

[62] `nox-mem-watcher` systemd service running `inotifywait` on `memory/` directories. See §3.1.

- **Recursion prevention**: `MEMORY.md` and `SESSION-STATE.md` are excluded to avoid feedback loops
- **Heartbeat**: Touch `/tmp/nox-mem-watcher-heartbeat` on every event for liveness monitoring

Upon trigger, `ingestFile()` executes:

1. Read file content with UTF-8 sanitization (fixes common mojibake patterns for Portuguese text)
2. Detect chunk type from relative file path
3. Extract date from filename pattern (YYYY-MM-DD)
4. Split content into semantic chunks:
   - Markdown: Split on H2/H3 headers, with sub-splitting for chunks exceeding 500 words
   - JSON: Array items become individual chunks; object entries become key-value pairs
   - Small chunks (<20 words) are merged with the previous chunk
5. Delete existing chunks for the same source file (idempotent re-ingestion)
6. Insert new chunks via prepared statement transaction
7. Auto-vectorize if GEMINI_API_KEY is available (up to 20 chunks per file)

3.2 Consolidation

Nightly consolidation (23:00, 5-minute stagger across agents) processes daily notes into structured topic files:

1. **Reindex**: Scan all `.md`/`.json` files in memory directories, rebuild chunk index
2. **Extract**: Use Ollama llama3.2:3b to identify facts, decisions, lessons, and action items from daily notes
3. **Append**: Add extracted content to topic files (decisions.md, lessons.md, people.md, projects.md, pending.md)
4. **Notion Sync**: Push structured items to "Memoria & Decisoes" Notion database (best-effort, non-blocking)
5. **Git Commit**: Auto-commit memory changes with standardized message format
6. **Session Update**: Refresh SESSION-STATE.md with current statistics

3.3 Deduplication

Before insertion, chunks are checked for duplicates using a two-tier strategy:

- **Primary**: Gemini cosine similarity with 0.85 threshold (when embeddings are available)
- **Fallback**: Keyword overlap calculation with 60% threshold
- **Audit**: Suppressed duplicates are logged to `dedup_log` table with reason and preview

3.4 Self-Evolution: How nox-mem Improves Over Time

Most memory systems decouple *retrieval* from *learning*: retrieval is online, learning is either absent or requires retraining a parametric backbone (MeMo) or relying on a closed evolution loop (EverOS EvoAgentBench [63]). nox-mem instead promotes self-evolution to a first-class subsystem composed of four cooperating mechanisms, all transparent and inspectable in the live SQLite file. This subsection is the direct counter to Gap #5 (no writeback) in Table 1.

3.4.1 Crystallize loop — `pending` → `lesson` after N hits, with pain accumulation The `crystallize` command (exposed via the CLI, the Model Context Protocol (MCP) server tool, and `POST /api/crystallize` + `POST /api/crystallize/validate` on the HTTP API; see the HTTP API server and the consolidation module [64]) implements an LLM-assisted consolidation that synthesizes durable entities from recent chunks. Operationally, chunks ingested into `memory/pending.md` (`chunk_type = pending`, `retention_days = 30`) act as a working set of unresolved items. As the same pending item is referenced by retrieval (and as related daily/team chunks accumulate around it), its effective *pain* signal grows — both via direct operator updates to the `pain` column (V9 schema) and via co-occurrence with high-pain neighbors. After enough hits and sufficient accumulated severity, `crystallize` uses Gemini to identify the pattern across chunks, emits a canonical entity file under `memory/entities/<type>/<slug>.md`, and effectively promotes the cluster from `pending` (30-day decay) to `lesson` (180-day decay) or `decision`/`project` (365-day decay) [65]. The promotion is wrapped in `withOpAudit()`, so the pre-state snapshot lives at `/var/backups/nox-mem/pre-op/crystallize-<ts>-<pid>-<uuid>.db` and a row lands in the append-only `ops_audit` table.

3.4.2 Pain weighting auto-adjust — feedback of use raises pain on hit chunks The `pain` field on `chunks` is an explicit severity in [0.1, 1.0] (0.1 trivial, 1.0 production outage). It can be set explicitly by the author, but it also drifts

[63] Hu, Gao, Zhou, Xu, Bai, Li, Zhang, Li, Zhang, Bing & Deng, *EverMemOS: A Self-Organizing Memory Operating System for Structured Long-Horizon Reasoning*, arXiv:2601.02163, 2026. Implementation: github.com/EverMind-AI (~5k stars, Apache 2.0). Publishes EverMemBench dataset and an EvoAgentBench-framed evolution loop. The only memory-OS peer in our taxonomy that publishes its own benchmark; §3.4 is the direct narrative counter to EvoAgent framing. Honest-comparison action item: run nox-mem on EverMemBench (queued as F4 in §7.2). This is also the system paper for what §6.3.3 measures — §6.3.3 reports a quality number **of** this system, and citing only its repository would leave the reader unable to check what was built. Its reported 93.05% LoCoMo / 83.00% LongMemEval are LLM-judged answer accuracy, a different quantity from the retrieval nDCG@10 of §6.3.3, and are not compared to it.

[64] HTTP handlers for `/api/crystallize` and `/api/crystallize/validate` in `staged/1.6/edits/api-server.ts:253-273`; core logic exports `crystallize()`, `validateProcedure()` and `listProcedures()`, wrapped by `withOpAudit()` (`staged/1.7a/edits/op-audit.ts`).

[65] V8 schema typed retention defaults (in `chunks.retention_days`): `feedback` = 0 (never-decay), `person` = 0 (never-decay), `lesson` = 180d, `decision` = 365d, `project` = 365d, `team` = 120d, `daily` = 90d, `pending` = 30d, `graph_node` = 60d, fallback = 90d. See `staged/1.7a/edits/salience.ts:46-56` (`DEFAULT_RETENTION_BY_TYPE`) and `CLAUDE.md` §"Schema v10".

upward implicitly: chunks that are *repeatedly* surfaced by retrieval and *accepted* by the operator (via the `feedback/` directory, whose `chunk_type = feedback` is never-decay) accumulate co-occurrence with high-pain entries during nightly consolidation, which feeds back into the salience formula. The result is that lessons learned from real incidents naturally rise to the top over time, while toy or low-severity content fades even when frequent. This is the inverse of pure recency- or frequency-based memory.

3.4.3 Salience decay continuous in background — `recency × pain × importance` The salience function lives in `staged/1.7a/edits/salience.ts` [66]. Its underlying principle is multiplicative — a memory scores high only when it is *simultaneously* recent, painful, and important:

`salience = recency × pain × importance`

with components:

- **recency** in [0,1] — half-life-style decay over the chunk's `retention_days` window (`feedback`/`person` → never-decay → `recency = 1.0`; everything else decays per Table V8 [67]). Decay is *continuous*: it is recomputed at every retrieval, so there is no batch step that "ages" memory — aging is a property of the read path.
- **pain** in [0,1] — severity as described in §3.4.2.
- **importance** in [0,1] — `chunk_type` / `source_type` / tier signal (manual mapping; e.g., `decision` and `lesson` rank above `daily`).

In production since Wave A (§5.1), this principle is realized as a **weighted-additive** form — `salience = W_IMPORTANCE·importance + W_RECENCY·recency + W_PAIN·pain + W_ACCESS·access_score` (weights 0.55 / 0.15 / 0.10 / 0.20) — which empirically outperformed the strict multiplicative product on the Wave A golden set; §5.1 reports the ablation. The multiplicative expression above states the principle; the additive form is what ships.

The mode of operation is gated by `NOX_SALIENCE_MODE`: `shadow` (default — compute and log to `/api/health.salience` but do not apply to retrieval rankings), `active` (apply as an additive delta in [-0.5, +0.5] on top of RRF), and `off` (short-circuit to 0 for ablation experiments). This three-state gate is the operational realization of *shadow discipline* (§4 and §5).

[66] `staged/1.7a/edits/salience.ts` — versioned implementation of the salience formula (multiplicative principle in §3.4.3; weighted-additive v2 production form `W_IMPORTANCE·importance + W_RECENCY·recency + W_PAIN·pain + W_ACCESS·access_score` in §5.1); lines 1–80 contain the module docstring spelling out the formula, the three-state mode gate, and the per-type retention defaults).

[67] V8 schema typed retention defaults (in `chunks.retention_days`): `feedback` = 0 (never-decay), `person` = 0 (never-decay), `lesson` = 180d, `decision` = 365d, `project` = 365d, `team` = 120d, `daily` = 90d, `pending` = 30d, `graph_node` = 60d, fallback = 90d. See `staged/1.7a/edits/salience.ts:46-56` (`DEFAULT_RETENTION_BY_TYPE`) and `CLAUDE.md` §"Schema v10".

3.4.4 Reflect pipeline — batch nightly cross-session synthesis The `reflect` command (CLI / MCP / `POST /api/reflect`, backed by the reflect module and cached via `getReflectCacheStats()` exposed in `/api/health.reflectCache` [68]) runs a batch synthesis pass over recent chunks. Its job is *not* to retrieve answers for the user but to produce cross-session lessons: it queries hybrid search for a topic, asks Gemini to summarize the patterns observed across the result set, and writes the summary back as a `feedback` or `lesson` chunk with the confidence default for derived chunks (`CONFIDENCE_DERIVED`). Because reflect results are themselves cached (LRU, exposed via `/api/health.reflectCache.entries`), expensive synthesis is amortized across repeated queries.

3.4.5 Consolidate — nightly orchestration that ties it together Nightly consolidation (23:00, 5-minute stagger across agents — see §3.2) is the orchestration layer that runs reflect for high-pain topics, runs crystallize for sufficiently aged `pending` items, and syncs the resulting durable entities to the topic files (`decisions.md`, `lessons.md`, `people.md`, `projects.md`). Like crystallize, it goes through `withOpAudit()` with a pre-op `VACUUM INTO` snapshot. This is the daily heartbeat of self-evolution.

Together, these four mechanisms make nox-mem a memory that *grows along the gradient of operator pain*: chunks that hurt the operator (production outages, lost work, repeated mistakes) survive decay, get promoted from pending to lesson, accumulate co-occurrence weight, and rank progressively higher — without retraining a model and without trusting a closed evolution loop. Every step is visible by opening `nox-mem.db` in `sqlite3` and inspecting `ops_audit`, `chunks.pain`, `chunks.retention_days`, and the entity files.

3.5 Session Priming: the read-side of self-evolution

The mechanisms above are *write-side* — they decide what the store keeps and how it ranks. But a memory that only grows is wasted if every new session starts blind and must re-discover context through cold search. **Session priming** closes the loop on the *read-side*: each session (any agent, any MCP/HTTP client) is born contextualized. `GET /api/brief` returns a salience-ranked, scope-filtered digest of the top-N chunks, injected at session start. The brief is strictly read-only over `chunks` — it never touches `access_count` or `last_accessed_at`, so the organic-use signal that feeds salience (§3.4) stays uncontaminated; serve-history is tracked in a separate `brief_log` table. With an `agent` scope the digest is a guaranteed union (~half agent-specific, half global high-salience), and near-duplicate variants are collapsed by token-containment so the budget is spent on distinct content.

[68] The reflect module exports `reflect()` and `getReflectCacheStats()` (wired in `staged/1.6/edits/api-server.ts:12`). Cache statistics are surfaced at `/api/health.reflectCache`.

An honest measurement, and a methodology caveat. Running priming in production (6 agents + shared workspace store, ~6.7k serves/day) surfaced a failure mode worth reporting. Over a clean 7-day window the brief served only **83 distinct chunks across 46.8k serves (0.18% diversity)**, median content age 48 days: salience, dominated by importance and the accumulated pain of old incidents, produces a near-static top set, and recently-written content (the very output the loop produces) rarely enters — we measured **931 recent (<=7d), relevant chunks (importance >= 0.7 or pain >= 0.7) that were never once served**. More instructive was a *negative* result on evaluation: we could not measure a downstream "follow-up rate" (does a primed chunk get used later?) because the signal does not exist — in 7 days there were **3 genuine searches and 0 `answer` calls**. This is structural, not a logging gap: priming-by-injection delivers the knowledge *into the context window*, so the agent does not re-query what it already holds. **The utility of an injection-based primer is therefore a property of the served set (diversity, freshness, coverage of what matters) — not of downstream re-retrieval.** Memory work that imports a re-query metric from RAG-style retrieval will mis-evaluate this class of mechanism.

Diversity term (D2). The fix follows the same discipline as §3.4.3: it does **not** fork `calculateSalience` (that would alter search too) — it is a re-rank *inside the brief only*. (A) a novelty penalty `brief_score = salience - min(P_max, lambda*log1p(n_serves))` reads `brief_log` to demote over-served chunks, saturating in log so serving 2,000x ~= serving 100x; (B) a freshness quota reserves F of the N slots for recent, relevant, not-yet-served content (a separate query — the candidate pool, ranked by importance and access, structurally excludes `access_count = 0` newcomers). A **high-pain floor** makes incidents with `pain >= 0.9` immune to both demotion and displacement: diversity must not hide the critical. Per the shadow-discipline rule (§3.4.3, §5), the term ships behind `NOX_BRIEF_DIVERSITY = off | shadow | active`, default off and bit-identical to the prior behavior; it is validated in shadow (logging the would-enter/would-leave diff against a per-day gate: diversity up, median age down, floor preserved, churn non-thrashing) before any flip to active. Notably, the shadow gate itself caught a design bug on its first run — the freshness quota was displacing `pain = 1.0` incidents — which the floor now prevents; the active-mode quality numbers are reported as ongoing validation (§7.2).

Active-mode validation surfaced a second, subtler failure — and the fix unifies the two slots. Two refinements followed the first gate. First, the freshness query as scoped (`global+agent`) only ever saw `sessions/<agent>/%`, never the curated `memory/entities/%` store, so freshly-consolidated lessons and decisions were structurally invisible to every brief; a *split-slot* interleave (one agent-recent plus one curated-global per brief, with its own age window) restored that channel. Second — and this is the instructive part — once the curated channel was live in `active`, a 24-hour gate showed the global slot serving **190 distinct curated chunks on day one, then exactly 1 per day thereafter**. The cause was the slot's *hard* anti-repeat (`id NOT IN brief_log`

over a 72h window): under production volume (~6.7k briefs/day) over a 189-chunk eligible pool, the entire pool is served — and thus excluded — within a single day, after which the slot dries up and fails open to the static salience top-1 for the remaining 72h. The first remedy *seemed* obvious: replace hard exclusion with the soft novelty penalty already used by the primary pool (mechanism A) — serving a curated chunk *demotes* it by `lambda*log1p(n_serves)` (capped at `P_max = 0.15`) rather than *removing* it, with the high-pain floor keeping critical incidents immune. Active-mode measurement refuted it: the 24-hour gate fell **146 → 67 → 3 distinct curated chunks per day** over three successive days. The cap was the flaw — `P_max` (0.15) is smaller than the base-salience gap between the pool's few high-salience outliers (decisions at importance 0.9 with high access counts) and its homogeneous body, so once the penalty saturated (within one 72h window under production volume) the pick reconverged to the static salience top-k. A bounded soft penalty cannot outvote an unbounded salience gap. The fix that held ranks the slot not by *count* of prior serves but by *time since last serve* (mechanism B', coverage-sampling): never-served first, then least-recently-served, tie-broken by salience. Having no ceiling, it sweeps the entire pool continuously regardless of the salience gap. The first full post-deploy 24-hour active gate (2026-06-27, ~6.8k briefs/day) confirmed it: **184 of 184 eligible entity files served (100% pool coverage), 190 distinct curated chunks, ~45 distinct chunks rotated per hour — sustained, not a day-one burst** (the cumulative distinct count reached 190 within four hours of deploy and held flat through hour 24; the pool is swept fast and then re-cycled by recency rather than exhausted). The general lesson — **hard de-duplication under volume » pool produces bursty rotation; a *saturating* soft penalty silently reconverges once its cap falls below the salience gap; only coverage-by-recency-of-serve, which has no ceiling, produces true continuous rotation** — is the kind of finding only a shadow→active→measure loop on real traffic exposes. An offline diversity metric on a static corpus would have scored all three designs identically, and the saturating-penalty regression in particular would have been invisible without a multi-day active gate.

---

4. Hybrid Search System

4.1 Architecture

Search combines three complementary retrieval methods:

Layer 1 — FTS5 BM25 (Keyword)

SQLite FTS5 with porter unicode61 tokenizer provides fast keyword matching. Results are scored using BM25 with column weights (chunk_text: 1.0, source_file: 0.5, chunk_type: 0.5). Post-retrieval boosting applies:

- Type boost: `decision` and `lesson` chunks receive 2.0x multiplier (higher

signal-to-noise ratio)
- Recency boost: Chunks from the last 7 days receive 1.5x multiplier

The query sanitizer strips special characters but preserves hyphens for compound terms (e.g., "nox-mem").

Layer 2 — Gemini Semantic (Vector)

Each chunk is embedded using Google's gemini-embedding-001 model[69] (3072 dimensions) with task type RETRIEVAL_DOCUMENT. Query embeddings use task type RETRIEVAL_QUERY for asymmetric similarity optimization.

Vectors are stored in sqlite-vec virtual tables. Retrieval uses cosine distance with a map table (vec_chunk_map) bridging vec_chunks rowids to chunks.id values due to sqlite-vec's rowid-only constraint.

Scoring normalizes distances to a 0-10 scale with type and recency boosting (1.5x and 1.2x respectively, lower than FTS5 to avoid double-boosting in fusion).

Layer 3 — Reciprocal Rank Fusion (RRF)

FTS5 and semantic results are merged using RRF with k=60:

```
RRF_score(d) = Σ 1/(k + rank_i(d))
```

Documents appearing in both result sets receive combined scores, marked as `match_type: "hybrid"`. Content-prefix deduplication (first 50 characters) prevents near-duplicate results.

4.2 Performance Characteristics

The hybrid approach provides significant quality improvements over single-method search:

Query	FTS5 Only	Hybrid	Analysis
"qual o proximo passo"	0 results	ROADMAP + PHASE-3	Semantic captures intent without keyword match
"nox-mem"	0 results	decisions.md + docs	Vector bypasses tokenizer hyphen issues
"quem e o [name]"	people.md	people.md + TEAM_MEMORY	RRF combines exact match + semantic context

The three queries are verbatim from production usage and are in Portuguese, the operator's working language; they are reported unchanged because the FTS5 and hybrid outcomes in the table were measured on these exact strings. In the third query a personal name occurring in the corpus is redacted as `[name]`; the retrieval behaviour reported for it was measured on the unredacted string.

[69] Google, *Gemini Embedding Model (`gemini-embedding-001`)*, 2024. ai.google.dev — model card. Cited in §3.3 for the 3072-dimension embedding used by Layer 2; a service reference, so the model card is the identifier.

4.3 Cross-Agent Search

The `crossSearch()` function opens all 7 databases in read-only mode, executes FTS5 queries in each, and merges results with agent attribution. Deduplication uses content-prefix comparison to handle shared documents that appear across multiple agent databases.

---

5. Empirical Evaluation

Headlines (2026-05-29/06-02, 5-batch canonical, four-benchmark triangulation + Q3 IterB ceiling break + Wave 2 backbone-conditional knob study): - **Q3 IterB ReAct — F_MH retrieval-stage ceiling broken:** on Gemini-3-flash bare baseline, multi-round retrieve-reason loop delivers **+2.01 pp clean F_MH lift (6.02% → 8.03%)** — exceeding the Wave A/B/C single-stage retrieval ceiling of 7.25 pp by +0.78 pp standalone, with the strongest backbone in the matrix (§5.5.2, dual-baseline reporting). - **Wave 2 backbone-conditional knob study:** Wave A single-stage retrieval knobs (KG path, AC, MQ) transfer at only **~24–40% efficiency** from gpt-4.1-mini to Gemini-3-flash. All three fail the >=+1.5 pp gate on Gemini-3-flash (NO-REPLICATE). 3-knob sum = +2.01 pp = 24% of original D74 projection +8.43 pp. **IterB ReAct remains the only validated F_MH lever on Gemini-3-flash** (§5.5.5). - **Wave 2 architectural lock discovery (NEW, D75):** IterB ReAct adapter deliberately short-circuits Wave A knobs via explicit `if not iterb_used_path:` guards in `adapter_nox_mem.py`. Composability requires explicit code patch, not merely env-var toggling. Composability projections must address both backbone-portability and architectural composability (§5.5.7). - **Wave 2 MQ MA backbone flip sub-finding:** MQ F_MH lift attenuates gpt-4.1-mini→Gemini (34% transfer rate), while MA composite **flips** from −1.38 pp regression on gpt-4.1-mini to +0.12 pp preserved on Gemini-3-flash. Multi-axis backbone-conditional behavior (§5.5.6). - **Wave 2 Capstone INDETERMINATE — infrastructure constraint, NOT scientific failure (draft):** IterB+KG+rerank composability test aborted after Hostinger VPS CPU steal 51–97% sustained. Batch 004 (n=49) preserved; 5-batch statistical threshold not reached. Capstone deferred to dedicated CPU infrastructure (§5.5.8). - **EverMemBench Overall (Gemini-3-flash, Backbone Matrix):** nox-mem **63.28%** vs MemOS 42.55% (Table 4 baseline, GPT-4.1-mini) = **+20.73 pp**; backbones differ, see §5.1.10. - **EverMemBench Memory Awareness composite (Gemini-3-flash):** nox-mem **88.42%** vs MemOS 55.68% (GPT-4.1-mini)

= **+32.74 pp**; backbones differ, see §5.1.10. - **MuSiQue-Ans**[70] **(multi-hop QA, classical):** dev answer F1 **58.62%** = **+22.82 pp vs IRCoT**[71] **35.80%** and **+8.92 pp vs EX(SA) 49.70%**; ~10 pp below Beam Retrieval[72] (69.2, test set) (§5.2.1). - **HotPotQA (multi-hop QA, classical):** dev distractor answer F1 **73.37%**, above the DPR+FiD reader range 65–72%; ~12 pp below Beam Retrieval (85.04) and FE2H (84.44), both blind test (§5.2.2). - **LoCoMo**[73] **retrieval@10 (cross-bench memory):** strict **74.52%**. Mem0's published 66.88% is an end-to-end **answer F1** and therefore not comparable to a retrieval@10 figure; our own F1 push is 51.85% (rank-5, above Zep/LangMem), constrained by a verbosity gap (§5.3.1–§5.3.2). - **Phase D (Gemini-2.5-flash):** nox-mem **62.22%** vs MemOS 59.27% = **+2.95 pp WIN**. - **Phase H v2 (GPT-4.1-mini cross-backbone):** nox-mem **51.68%** vs MemOS 42.55% = **+9.13 pp WIN** (95% CI [49.88, 53.49]). - **Backbone portability:** nox-mem regresses −10.54 pp on Gemini-2.5-flash → GPT-4.1-mini swap vs MemOS −16.72 pp = **1.6× more portable** (§5.8). - **Wave B/C composability (D68 + D69):** KG+MAP additive on F_MH +4.04 pp (different-stage compose); same-stage retrieval knobs cap at ~7.25 pp F_MH (Wave C ceiling). - **EverMemBench F_MH paradox REFINED:** MuSiQue 58.62% F1 and HotPotQA 73.37% answer F1 — above their conventional reference readers, roughly 10–12 pp below current SOTA (Beam Retrieval) — show multi-hop composition is not where this pipeline fails, which rules out a wholesale reasoning failure as the explanation for F_MH; the difficulty of the track itself is visible same-metric, with the best published system (MemOS) at **18.88%** strict EM. EverMemBench F_MH 3–7% is therefore attributed principally to the task setup (very long conversation chains + strict scoring), and MAS orchestration (Q3 IterB ReAct, this revision) adds **+2 pp on top of strongest backbone** above the retrieval ceiling (§5.4, §5.5). - **Operational (self-hosted):** KG path p50 **2.9 ms**, **$0/query**, **399 MB RSS** self-hosted single-process (§5.7). - **Methodology:** all claims use the **5-batch + 95% CI canonical protocol**. Single-batch overstates effects 3–6×. - **Dual-baseline honest reporting (D74):** orchestration-mechanism claims report against

[70]Trivedi, Balasubramanian, Khot & Sabharwal, *MuSiQue: Multihop Questions via Single-hop Question Composition*, TACL 2022. arXiv:2108.00573. The multi-hop QA dataset used in §5.2.

[71]Trivedi, Balasubramanian, Khot & Sabharwal, *Interleaving Retrieval with Chain-of-Thought Reasoning for Knowledge-Intensive Multi-Step Questions*, ACL 2023. arXiv:2212.10509. The IRCoT baseline compared against in §5.2.

[72]Zhang, Zhang, Zhang, Liu & Huang, *End-to-End Beam Retrieval for Multi-Hop Question Answering*, NAACL 2024. arXiv:2308.08973. The MuSiQue leaderboard system referenced as the upper bound in §5.2.1.

[73]Maharana, Lee, Tulyakov, Bansal, Barbieri & Fang, *Evaluating Very Long-Term Conversational Memory of LLM Agents*, ACL 2024. aclanthology.org/2024.acl-long.747. The LoCoMo benchmark used in §5.5 and §6.

both (a) project-convention baseline (Phase H v2 GPT-4.1-mini, conflates backbone + mechanism) and (b) the strongest in-matrix bare baseline (Gemini-3-flash, isolates mechanism). Ceiling-break claims load on (b).

This section documents three evaluation tracks that triangulate the same architectural claims from complementary directions: the **Wave A ablation series** (§5.1.1–§5.1.4, entity-flavored golden set, nDCG@10) exercising the V10 schema's section/source-type/salience drivers; the **EverMemBench cross-system series** (§5.1.5–§5.1.10, n=3,121 queries, task-accuracy vs MemOS Table 4 baselines[74]) covering Phase D / H v2 / G / Lab Q1 standalone knobs / Wave B/C composability / Backbone Matrix; and the **classical multi-hop QA series** (§5.2, MuSiQue + HotPotQA) bounding multi-hop reasoning as competent on standard benchmarks, above their conventional reference readers and below current SOTA where corpus structure and scoring are not adversarial. §5.3 cross-validates on LoCoMo (memory-bench, conversational), §5.4 resolves the EverMemBench F_MH paradox using the §5.2 and §5.3 evidence, §5.5 reports Q3 orchestration mechanism-class findings — including the **Q3 IterB ReAct ceiling break** (§5.5.2), the **Wave 2 backbone-conditional knob portability study** (§5.5.5–§5.5.6), the **architectural composability lock discovery** (§5.5.7), and the **Wave 2 capstone deferral** (§5.5.8) — §5.6 cross-validates on LongMemEval, §5.7 reports operational characteristics (latency / cost / footprint), §5.8 documents the methodology and honest limitations.

Dual-baseline reporting convention (D74, 2026-05-31). Orchestration-mechanism evaluations in this revision report against **two** baselines: (a) the project-convention Phase H v2 GPT-4.1-mini baseline, which preserves comparability with prior revisions but **conflates mechanism lift with any backbone swap**; and (b) the strongest in-matrix bare baseline (Gemini-3-flash, §5.1.10), which **isolates the mechanism's clean effect** on the best available reasoning substrate. Any claim of breaking the retrieval-stage F_MH ceiling load-bears on the (b) clean number — reporting only (a) would conflate ReAct mechanism lift with the +20.73 pp Overall and +32.74 pp MA composite lifts that the Gemini-3-flash backbone already provides standalone (§5.1.10). The convention applies forward to §5.5 Q3 IterB (this revision) and any future MAS-class orchestration evaluations.

5.1 EverMemBench + Wave A ablation series

5.1.1 Wave A ablation — setup and headline The Wave A evaluation uses an entity-flavored golden set of 100 queries (`entity-eval.db`), curated from production usage to exercise the V10 schema's `section` and `pain`

[74] **"MemOS Table 4" throughout §5 means the MemOS row of Table 4 in the EverMemBench paper**[^longhorizon] — *not* a table in the MemOS paper itself. The MemOS system is Li, Xi, Li, Chen, Chen, Song, Niu, Wang *et al.*, *MemOS: A Memory OS for AI System*, arXiv:2507.03724, 2025; we compare against its published numbers and did not re-run it (§5.8.6).

dimensions. Configurations are toggled via environment-variable feature gates (`NOX_SALIENCE_MODE`, `NOX_DISABLE_TIER_BOOST`, `NOX_ENABLE_TIER_BOOST`, `NOX_DISABLE_SECTION_BOOST`, etc.), allowing isolation of individual ranking components without code changes between runs. All measurements occur after deployment of the salience formula with `tier_boost` off by default, the `source_type` backfill of 67,949 chunks, and the search wiring. Reported nDCG@10 follows the standard TREC formulation (gain by relevance, log-position discount).

> **Wave A headline (canonical, 2026-05-19):** A8 full stack with active salience reaches **nDCG@10 = 0.6237** on the entity-flavored golden set (n=100), a **+78.8% relative improvement over the G3 baseline (0.3488)** measured prior to Wave A deployment, and **+9.4% over the mid-deployment G4 checkpoint (0.5702)**. The peak ablation isolating `section_boost` alone (A3) reaches 0.6228, recovering **99.85% of the full stack** — section-aware ranking is the dominant driver of the lift.

Progression vs prior ablation generations:

Generation	Date	A8 nDCG@10	Δ vs G3 baseline	Notes
G3 baseline (pre-Wave A)	2026-05-15	0.3488	—	Multiplicative salience, tier_boost on, section_boost only via legacy code path
G4 mid-deployment	2026-05-18	0.5702	+63.5%	Additive salience wired but *active* < *shadow* puzzle observed
G5 V3 canonical	**2026-05-19**	**0.6237**	**+78.8%**	Wave A fully deployed; reversal *active* > *shadow* confirmed

The four sub-claims below decompose the +78.8% total into measurable contributions; the full G5 V3 matrix (12 configurations) is archived under `audits/`.

5.1.2 Wave A — Claim 1: Additive salience outperforms multiplicative The Wave A formula replaces the legacy multiplicative `salience = recency × pain × importance` with a weighted-additive form:

```
salience = W_IMPORTANCE·importance + W_RECENCY·recency
         + W_PAIN·pain + W_ACCESS·access_score
W_IMPORTANCE = 0.55  W_RECENCY = 0.15  W_PAIN = 0.10  W_ACCESS = 0.20
```

Result. With `NOX_SALIENCE_MODE=active`, A8 reaches 0.6237 vs. 0.6155 with `shadow` (A7) — a +1.3% lift and the reversal of the G4 puzzle, where shadow

had outranked active. The multiplicative form concentrated 99.7% of chunks in the [0.05, 0.40] salience range, dominated by 90.67% of chunks at the default `pain = 0.2` and 99.76% of chunks with `recency in [7, 30]` days; small differences in any factor were swallowed by the product. The additive form exposes each dimension proportionally to its calibrated weight, preserving signal from pain spikes and importance heuristics without requiring all three factors to be simultaneously non-default.

5.1.3 Wave A — Claim 2: `section_boost` is the moat (99.85% of the gain) Isolating `section_boost` alone (A3 ablation: section enabled, tier off, source_type off, salience shadow) yields **nDCG@10 = 0.6228 = 99.85% of A8's full-stack 0.6237**. The V10 schema multipliers — `compiled = 2.0`, `frontmatter = 1.5`, `timeline = 0.8`, legacy = 1.0 — together with the entity-file format introduced in v3.7 explain the majority of the headline improvement. The format's scale, measured 2026-09-09: **865 section-bearing chunks** across **239 distinct source files** represented in the store, of which **184 still exist on disk** (a deleted source file leaves its chunks behind), averaging **3.62 sections per file** rather than the uniform 3 the format suggests, since `timeline` contributes 1..N. An earlier figure of "769 entity files × 3 sections ~ 2,307 chunks" appeared here and is not re-verifiable against any dated measurement; it also contradicted the "184 of 184 eligible entity files" reported in §4.3. Census script and artifact: `paper/measurement/censo-corpus.py`.

The negative control A11 (full stack minus `section_boost`) drops to 0.5646, **−9.5% relative to A8**, confirming the contribution is not redundant with semantic embeddings or RRF fusion. This is the architectural pivot the paper's narrative rests on: section-aware boosting over an entity-file canonical form is the load-bearing component, not the multiplicative salience formula that the v1 paper draft over-emphasized.

5.1.4 Wave A — Claims 3 & 4: `tier_boost` and `source_type` calibration `tier_boost` and `source_type` calibrate rather than drive; neither carries the gain that §5.1.3 attributes to `section_boost`. Detail in the anonymized supplementary material (§S5.1.4.)

5.1.5 EverMemBench Phase D — Gemini-2.5-flash headline (5-batch) **Config:** phaseB adapter, top_k=20, rerank OFF, Gemini-2.5-flash backbone. Evaluation on EverMemBench (EverOS canonical benchmark), 5-batch canonical set (batches 004, 005, 010, 011, 016), n=3,121 total queries.

Metric	nox-mem (5-batch)	MemOS Table 4 (Gemini column)	Δ
Overall accuracy	**62.22%**	59.27%	**+2.95 pp WIN**

The 5-batch methodology (§5.8) is canonical for this claim. Single-batch estimates from prior runs showed higher variance; the 5-batch aggregate is the

defensible number for any comparative claim. The phaseB adapter uses the production hybrid search stack (FTS5 + Gemini-embedding-001 3072d + RRF k=60) with no generation-side augmentation, so the win reflects pure retrieval-quality advantage.

5.1.6 EverMemBench Phase H v2 — GPT-4.1-mini cross-backbone (5-batch) Config: phaseB adapter, top_k=20, rerank OFF, GPT-4.1-mini backbone (OpenAI). 5-batch, n=3,121.

Metric	nox-mem 5-batch	95% CI (t-dist, n=5)	MemOS Table 4 (GPT-4.1-mini col)	Δ
Overall	**51.68%**	[49.88, 53.49]	42.55%	**+9.13 pp WIN**
F_MH (multi-hop)	~3–5%	wide	18.88%	−13 to −16 pp gap
MA_C (Memory Constancy)	84.60%	—	69.90%	+14.70 pp
MA_P (Memory Proactivity)	65.40%	—	51.99%	+13.41 pp
MA_U (Memory Update)	70.03%	—	45.15%	+24.88 pp

Sub-dim summary: 7/9 sub-dimensions WIN. F_MH and F_TP are the only losses. The 95% CI lower bound (49.88%) strictly exceeds MemOS (42.55%), confirming the win is robust to per-batch variance.

Outlier detection. Batch 004 alone reported 54.15% overall (+11.60 pp vs MemOS), which was a +1.70sigma upper-tail outlier (per-batch distribution: 004=54.15%, 005=50.82%, 010=50.72%, 011=50.87%, 016=51.83%, stdev=1.45 pp). Without 5-batch validation, the single-batch headline would have overstated the advantage by 1.27×. This is the canonical illustration of why the 5-batch protocol exists (§5.8).

5.1.7 EverMemBench Phase G — Cross-encoder rerank trade-off study (5-batch) Config: MiniLM[75]-L-6-v2 cross-encoder rerank (22M params), top_k=20 pool rescored, Gemini-2.5-flash backbone. 5-batch, n=3,121.

[75]Wang, Wei, Dong, Bao, Yang & Zhou, *MiniLM: Deep Self-Attention Distillation for Task-Agnostic Compression of Pre-Trained Transformers*, NeurIPS 2020. arXiv:2002.10957. The cross-encoder checkpoint reranked in §5.1.7 and §5.1.9 is the `ms-marco-MiniLM-L-6-v2` distillation of this model (22M params).

Cross-encoder reranking[76] exposes a **4-dimensional trade-off** across retrieval workload types:

Category type	Δ vs Phase D (no rerank)	Direction
Hard-recall: F_MH (multi-hop)	**+1.61 pp** (95% CI [3.97, 9.69] — overlaps baseline)	marginal gain
Hard-recall: F_HL (high-level)	+2.58 pp	marginal gain
Hard-recall: F_TP (temporal)	+2.00 pp	marginal gain
Head-precision: F_SH (single-hop)	+0.40 pp	quasi-neutral
Head-precision: MC (multi-choice)	−2.63 pp	regression
Memory Awareness: MA_C	**−4.00 pp**	significant regression
Memory Awareness: MA_P	**−2.80 pp**	significant regression
Memory Awareness: MA_U	**−3.84 pp**	significant regression
Overall	−0.96 pp	net regression

The F_MH gain of +1.61 pp closes only **11.7% of the MemOS F_MH gap** (Phase D baseline 5.22% → Phase G 6.83% vs MemOS 18.94%). The Memory Awareness (MA) regression of −3 to −4 pp was invisible in the single-batch gate (batch 004) due to selection bias — batch 004 already had the lowest MA performance of the five batches, masking the cost. The −0.96 pp overall regression is real across all 5 batches (2.3× smaller than the single-batch −2.24 pp estimate, but consistent in direction).

Verdict: REJECT as default. Ship opt-in via `--rerank` flag / `NOX_RERANKER_ENABLED=1` / `/api/answer?mode=exploratory`. Documented latency cost: +3.7 s p50. Workloads with known multi-hop-heavy profiles and tolerance for MA regression may benefit; all other workloads do not.

Wave B and Wave C compose additively on F_MH; the ceiling is the retrieval stage, not the orchestrator. Detail in the anonymized supplementary material (§S5.1.9.)

5.1.8 EverMemBench Lab Q1 — Retrieval augmentation standalone knobs (5-batch) Three retrieval-augmentation knobs (KG expansion, adjacent-chunk inclusion, multi-query fan-out) were measured standalone on the 5-batch protocol. Each is individually small and none reaches significance alone; the per-knob table and the per-backbone interaction matrix are in the anonymized supplementary material. What the main text uses from this study is the composability result of §5.1.9 and the backbone-conditional behaviour of §5.5.5–§5.5.6.

[76]Reimers & Gurevych, *Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks*, EMNLP 2019. arXiv:1908.10084. Source of the bi-encoder / cross-encoder distinction used throughout §5.1.7 and §7.2 F5.

5.1.9 Wave B + Wave C composability — additive F_MH and the retrieval-stage ceiling The Lab Q1 standalones identified four orthogonal mechanisms with overlapping F_MH lift profiles. Wave B (D68) and Wave C (D69) measure composability — do they stack, or do they overlap?

D68 KG + MQ co-fire analysis (same-stage retrieval): KG path and MQ expansion overlap at **90.8% co-fire rate** on EverMemBench queries — both activate on the same query population (entity-bearing multi-hop queries). Composability is non-additive on overlapping queries; net F_MH lift KG+MQ = +3.93 pp (vs predicted +6.42 pp), confirming the overlap.

D68 KG + MAP composability (different-stage): KG (entity-walk) and MAP (section bypass) operate at different retrieval stages and compose additively:

Configuration	F_MH	Δ vs Phase H v2
Phase H v2 baseline	3.21%	—
Phase KG standalone	6.02%	+2.81 pp
Phase MAP standalone	~7.23%	+4.02 pp
Phase KG+MAP composed	**7.25%**	**+4.04 pp (additive on F_MH)**

KG+MAP closes ~**24% of the MemOS F_MH gap** while staying within MA tolerance on multi-stage composition (MAP MA cost is partially absorbed when KG entity-walk pre-filters profile queries).

D69 Wave C ceiling — same-stage retrieval triple compose: Wave C tested KG + MQ + MAP triple composition with CLEAN refinement (sequential 5-batch + outlier-aware aggregation). Result: triple composition caps at ~**7.25 pp F_MH**, **statistically indistinguishable from KG+MAP doublet**. Adding MQ on top of KG+MAP yields no incremental lift. The interpretation: retrieval-stage knobs have a structural ceiling near +7.25 pp F_MH against the EverMemBench corpus — further gains require either orchestration-stage mechanisms (Q3, §5.5) or backbone upgrades (§5.1.10).

Composability triangulation summary (D64-D69):

1. Same-stage retrieval knobs **overlap** (D68: KG + MQ 90.8% co-fire) — composing them does not add proportionally.
2. Different-stage knobs **compose additively** on F_MH (D68: KG + MAP +4.04 pp combined).
3. Retrieval-stage stacking **caps at ~+7.25 pp F_MH** (D69 Wave C ceiling) — further F_MH gain requires moving up the stack.
4. Q3 orchestration is the open path for incremental F_MH gain beyond the retrieval ceiling (§5.5).

5.1.10 Backbone Matrix — Gemini-3-flash on EverMemBench, above the published MemOS numbers **Config:** phaseB adapter, top_k=20,

rerank OFF, Gemini-3-flash backbone (frontier reasoning tier). 5-batch n=3,121.

Metric	nox-mem (Gemini-3-flash)	MemOS Table 4 baseline (GPT-4.1-mini col)	Δ vs MemOS	Δ vs nox-mem gpt-4.1-mini (Phase H v2)
Overall	**63.28%**	42.55%	**+20.73 pp**	+11.60 pp
MA	**88.42%**	55.68%	**+32.74 pp**	+15.08 pp
composite				
MA_C	~95%	69.90%	+25 pp class	+10 pp class
MA_P	~83%	51.99%	+31 pp class	+18 pp class
MA_U	~87%	45.15%	+42 pp class	+17 pp class

Gemini-3-flash leads on both the Overall and Memory Awareness composite tracks. The MA composite at **+32.74 pp over the published MemOS numbers** is consistent with a structural advantage from the V10 schema's section/source-type/salience drivers when paired with a frontier-tier reasoning backbone.

The backbones differ, and the deltas are not SOTA claims. The MemOS column is the published Table 4 result obtained on **GPT-4.1-mini**; our column is **Gemini-3-flash**. Beating a published number produced on a weaker backbone is a cross-backbone comparison, not a state-of-the-art result, and §5.5.4–§5.5.8 measure directly how much backbone choice alone can move these metrics (single-stage retrieval knobs transfer at only 0–40% between these two backbones). The split-matched comparison against MemOS is the GPT-4.1-mini row in §5.1.6 (+9.13 pp, 95% CI [49.88, 53.49]); that one holds the backbone fixed and is the number to cite when the question is architecture rather than backbone.

Backbone Matrix interpretation. The +20.73 pp Overall and +32.74 pp MA composite lifts vs gpt-4.1-mini baseline are not exclusively backbone-driven: nox-mem's V10 retrieval stack contributes ~+9.13 pp Overall and ~+25 pp MA composite at the gpt-4.1-mini tier alone (Phase H v2, §5.1.6). The incremental +11.60 pp Overall and +15.08 pp MA composite from the backbone swap reflect Gemini-3-flash's superior reasoning over retrieved evidence — the architecture and backbone compose multiplicatively, not additively.

5.1.11–5.1.12 Cross-backbone analysis and the F_MH retrieval-bound finding Reported in full in the anonymized supplementary material (§S5.1.11, §S5.1.12): backbone-portability analysis and the gpt-4.1-mini-era strategic implication of the F_MH retrieval-bound result. Neither is referenced elsewhere in this manuscript; the conclusion both feed — that the EverMemBench F_MH gap is retrieval-bound rather than a reasoning deficit — is stated in §5.4 and used in §7. ### 5.2 Classical multi-hop QA — competitive without fine-tuning, below current SOTA

The EverMemBench F_MH gap raised an open question: is nox-mem's multi-hop reasoning genuinely limited, or is the EverMemBench F_MH track exposing a corpus-specific structural challenge? To answer this directly, we ran nox-mem against two canonical multi-hop QA benchmarks where the task structure is well-known and reader SOTA numbers are published: **MuSiQue** (multi-hop questions decomposable into sub-questions) and **HotPotQA** (multi-hop questions over Wikipedia with distractor paragraphs). Both are textbook adversarial multi-hop setups; both are widely-used reference benchmarks for retrieval-augmented multi-hop systems.

The result: on both benchmarks nox-mem performs multi-hop QA competently without any benchmark-specific fine-tuning, landing above the specialized multi-hop readers conventionally used as reference points for each dataset, and below the current state of the art. The purpose here is diagnostic — establishing whether multi-hop composition is a capability floor for the system — not a leaderboard claim.

5.2.1 MuSiQue-Ans dev — answer F1 58.62%, above EX(SA), below Beam Retrieval **Config:** nox-mem hybrid retrieval (FTS5 + Gemini-embedding-001 + RRF, top_k=20), GPT-4.1-mini generation backbone, MuSiQue dev set with full paragraph corpus. Per-question metric: F1 over tokenized answer match.

System	Split	Answer F1	Δ vs nox-mem	Source
Beam Retrieval (Zhang et al., NAACL 2024)	**test**	**69.20%**	**+10.58 pp**	Table 4 (arxiv:2308.08973)
nox-mem (hybrid, no rerank)	**dev**	**58.62%**	—	this work (§5.2.1)
EX(SA) (Trivedi et al. 2022)	dev	49.70%	−8.92 pp	MuSiQue paper (arxiv:2108.00573)
IRCoT (Trivedi et al. 2023)	dev	35.80%	−22.82 pp	IRCoT paper (arxiv:2212.10509)

Splits differ in the top row. Beam Retrieval reports answer F1 on the MuSiQue-Ans *test* set (their Table 4); its development-set table reports *retrieval* EM/F1 (77.37/79.31), not answer F1, so no split-matched answer figure is published for that system. Dev/test variance on this benchmark is on the order of one point, well inside the ~10-point gap, but the mismatch is stated rather than elided.

nox-mem lands above both readers published with the benchmark, and 10.58 pp below Beam Retrieval, the current published state of the art on this metric. Two cautions apply to the margins over EX(SA) and IRCoT. First, both are 2022–2023 systems while our reader is GPT-4.1-mini: the comparison confounds retrieval design with three years of backbone progress, and we do not claim the

delta measures the former. Second, EX(SA) at 49.70 is essentially the baseline Beam Retrieval itself improved upon (49.0 → 69.2), so "strongest specialized reader in the MuSiQue paper" is a statement about that paper, not about the field. Protocol also differs: Beam Retrieval selects from a per-question candidate set of 10–20 passages, while we retrieve top_k=20 over the full paragraph corpus. What the number does support is narrower and sufficient for §5.4 — multi-hop composition is not a capability floor for this pipeline. Two structural factors contribute:

1. **Hybrid retrieval recall at top_k=20** vs IRCoT's iterative CoT-retrieval loop (lower recall ceiling per round).
2. **RRF fusion of FTS5 + dense embeddings** delivers diverse candidate paragraphs from both lexical and semantic similarity tracks, increasing the probability that all multi-hop bridges are in the candidate pool.

Per-hop and per-type breakdowns confirm the gain is broad (not driven by a single hop count or question template); detailed numbers in `eval/musique/RESULTS-MUSIQUE.md`.

5.2.2 HotPotQA distractor — answer F1 73.37%, above DPR+FiD, below Beam Retrieval and FE2H **Config:** nox-mem hybrid retrieval (same config as §5.2.1), GPT-4.1-mini generation backbone, HotPotQA dev distractor[77] full corpus. Per-question metric: ans_F1 over tokenized answer match.

System	Split	Answer F1	Δ vs nox-mem	Source
Beam Retrieval (Zhang et al., NAACL 2024)	**blind test**	**85.04%**	**+11.67 pp**	Table 5 (arxiv:2308.08973)
FE2H on ALBERT	**blind test**	84.44%	+11.07 pp	hotpotqa.github.io leaderboard
nox-mem (hybrid, no rerank)	**dev**	**73.37%**	—	`audits/` HotPotQA dev run
DPR+FiD reader (range, published)	dev	65–72%	−1.37 to −8.37 pp	DPR (arxiv:2004.04906) + FiD (arxiv:2007.01282)
BERT reader, original dataset baseline (Yang et al. 2018)	dev	~58%	−15+ pp	HotPotQA paper (arxiv:1809.09600)

Splits differ in the top two rows. The official HotPotQA leaderboard reports the blind test set only; our figure is dev distractor. As with §5.2.1, the mismatch is smaller than the gap by an order of magnitude, and is stated rather than elided.

[77] Yang, Qi, Zhang, Bengio, Cohen, Salakhutdinov & Manning, *HotpotQA: A Dataset for Diverse, Explainable Multi-hop Question Answering*, EMNLP 2018. arXiv:1809.09600. Used in §5.2.2, §5.4.

The answer F1 of 73.37% sits above the DPR+FiD reader range without specialized training or HotPotQA-specific fine-tuning, and roughly 11.7 points below the leaderboard's leading entries. Note that DPR (2020) and FiD (2021) are later general-purpose systems, not the reader published with HotPotQA in 2018 — that baseline is the BERT reader at ~58%. The result corroborates §5.2.1 in the narrow sense that matters for §5.4: general-purpose hybrid retrieval with an off-the-shelf reader clears classical multi-hop QA competently, well short of purpose-built systems that optimize the retrieval chain end-to-end for these datasets.

5.2.3 What the classical-QA results do and do not license for the F_MH narrative The MuSiQue and HotPotQA results establish a critical decoupling: - **Multi-hop reasoning quality** is a property of the reader (generation backbone) plus the retrieval candidate pool. - **EverMemBench F_MH** measures something different — section-anchored entity-chain composition over very long conversation histories, with strict exact-match scoring.

On benchmarks where multi-hop reasoning quality is the question and corpus structure is friendly to general-purpose hybrid retrieval (MuSiQue, HotPotQA), nox-mem performs in the same regime as published specialized systems without matching the leaders. That licenses a bounded claim — multi-hop composition is not where this pipeline fails — and not an unbounded one. The 10–12 point shortfall against Beam Retrieval means classical multi-hop still has headroom for this system too, so the EverMemBench F_MH gap cannot be attributed to task structure alone on the strength of these two results. The same-metric evidence that the F_MH track is intrinsically hard is separate and stronger: the best published system on it reaches 18.88% strict EM. §5.4 develops what remains.

5.3 LoCoMo cross-bench — retrieval ceiling; F1 constrained competitive

LoCoMo (Maharana et al. 2024; arxiv:2402.17753) is the canonical long-conversation memory benchmark with 10-session dialogues and human-annotated multi-session question-answer pairs. It provides both a retrieval metric and an end-to-end F1 metric; Mem0 reports SOTA F1 66.88% on its own published baseline. nox-mem was evaluated on both tracks with hybrid retrieval + GPT-4.1-mini.

5.3.1 Retrieval@10 ceiling — strict 74.52% (not comparable to Mem0's answer-F1) Config: nox-mem hybrid retrieval, top_k=10, oracle-free, full LoCoMo dev corpus.

Track	nox-mem (strict)	nox-mem (adjacency-2)	Mem0 published F1	note
Overall retrieval@10	**74.52%**	87.10%	66.88% (F1)	different metric, not a head-to-head
Multi-hop retrieval@10	**82.21%**	**92.91%**	—	—
Single-hop retrieval@10	71.40%	84.13%	—	—
Temporal retrieval@10	68.94%	82.31%	—	—

The strict retrieval@10 of 74.52% is not comparable to Mem0's published end-to-end F1 of 66.88% (retrieval@10 vs answer-F1 are different metrics, not a same-corpus head-to-head; for the like-for-like nDCG@10 comparison where Mem0 wins LoCoMo under its native embedder, see §6.3). This is the retrieval **ceiling** for the corpus — the best a prompt-level F1 push can hope for from candidates retrieved at top_k=10. The multi-hop sub-track at 82.21% strict / 92.91% adjacency-2 confirms that multi-hop retrieval on LoCoMo is structurally easier than on EverMemBench (consistent with the task-setup account proposed in §5.4, which we do not establish as the whole explanation).

5.3.2 F1 push — 51.85% rank-5 (above Zep / LangMem, below Mem0 SOTA) A prompt-level F1 push with the same retrieval pool was measured to quantify the gap between retrieval ceiling and end-to-end F1.

System	LoCoMo F1	Rank	Notes
Mem0 SOTA (published)	66.88%	1	Per Mem0 paper
Mem0 (replication)	~60%	2	Range from internal estimate
OpenAI-memory (published)	~55%	3	Estimated from Mem0 comparison table
LangGraph (published)	~52%	4	Estimated
nox-mem (F1 push)	**51.85%**	**5**	**Above Zep 50.40% / LangMem 50.21%**
Zep (replication)	50.40%	6	Internal measurement
LangMem (replication)	50.21%	7	Internal measurement

nox-mem's F1 push of 51.85% is rank-5 across the benchmarked systems, **above Zep and LangMem** but below Mem0 SOTA. The gap between retrieval ceiling (74.52%) and F1 push (51.85%) is the **verbosity gap** — F1 scoring penalises verbose answers that contain the correct fact embedded in longer responses.

Mem0's specialized fact-extraction prompts close this gap; nox-mem's general-purpose retrieval + GPT-4.1-mini prompt does not.

The date-normalization knob contributed +2.8 pp F1 on temporal sub-track by aligning date format expressions between query and corpus chunks; this is the largest single tunable knob for LoCoMo F1.

5.3.3 Path to >=55% F1 — composition orchestration (Q3) Route to >=55% F1 is orchestration composition, not a retrieval change. Detail in the anonymized supplementary material (§S5.3.3.)

5.4 EverMemBench F_MH paradox — resolved

5.4 EverMemBench F_MH paradox — resolved

The triangulation of three independent multi-hop measurements forces a reframing of the EverMemBench F_MH absolute number:

Benchmark	Multi-hop metric	nox-mem score	Conventional reference readers	Published SOTA (split noted)	Verdict
MuSiQue-Ans dev	answer F1 (multi-hop decomposable)	**58.62%**	35.80% (IRCoT) – 49.70% (EX(SA))	69.20% (Beam Retrieval, *test*)	+8.92 pp vs EX(SA); **−10.58 pp vs SOTA**
HotPotQA dev distractor	answer F1 (multi-hop bridge)	**73.37%**	65–72% (DPR+FiD)	85.04% (Beam Retrieval, *blind test*)	above DPR+FiD; **−11.67 pp vs SOTA**
LoCoMo dev	retrieval@10 strict	**74.52%**	66.88% (Mem0 answer-F1, different metric)	**retrieval ceiling; not comparable to Mem0 F1**	
LoCoMo dev (F1 push)	F1 (verbosity-sensitive)	51.85%	66.88% (Mem0 SOTA)	rank-5, verbosity gap	
EverMemBench F_MH	strict EM (multi-hop chain on long conv)	3–7% (5-batch)	18.88% (MemOS Table 4)	**−13 to −16 pp gap**	

If nox-mem's multi-hop reasoning were structurally weak, the MuSiQue and HotPotQA results would sit near the original benchmark readers rather than well above them. They do not: the pipeline composes multi-hop answers competently on both. That rules out a wholesale reasoning failure as the explanation for F_MH, which is what this argument requires. It does **not** establish that classical multi-hop is saturated for this system — the 10–12 point shortfall against Beam Retrieval says the opposite — so the contrast with EverMemBench F_MH is a contrast between *competent-with-headroom* and *3–7%*, not between *solved* and *broken*. The quantitative evidence that the F_MH track is

intrinsically hard does not come from cross-metric comparison at all: it is the 18.88% strict EM that the best published system on this same track and same metric attains. The LoCoMo retrieval ceiling at 74.52% strict (82.21% multi-hop sub-track) further confirms that multi-hop retrieval over long conversations is achievable.

The proposed explanation. Four features of the EverMemBench task setup are candidates for the F_MH 3–7% number, and we present them as the leading account rather than an established attribution — nothing below separates a corpus effect from a system limitation, and §5.2 leaves 10–12 points of headroom on classical multi-hop:

1. **Very long conversation chains.** EverMemBench F_MH questions require composing facts across 100+ conversation turns, far longer than MuSiQue (<=4 paragraphs) or HotPotQA (2 bridge paragraphs).
2. **Strict scoring.** EverMemBench F_MH uses strict exact-match against canonical answers; minor wording variations are penalised even when the answer is correct. MuSiQue F1 and HotPotQA ans_F1 are partial-credit scores.
3. **Entity-anchor sparsity.** EverMemBench questions often lack explicit entity tokens that nox-mem's section/source-type boost framework can latch onto. The MAP (bypass-entity) mechanism of the Lab Q1 knob series (§5.1.8) was designed specifically to address this sparsity.
4. **Memory-vs-retrieval mismatch.** EverMemBench is a *memory* benchmark with implicit world-state updates; the chunks that answer F_MH questions may not be the chunks that explicit retrieval would surface. This is the architectural distinction MemOS optimises for.

Implication for Q3 priorities — refined by D74 (2026-05-31). The §5.4 framing shifts Q3 retrieval-mechanism priorities: pure retrieval-stage knobs (KG, MQ, MAP) cap at ~+7.25 pp F_MH (Wave C ceiling §5.1.9). Closing the remaining EverMemBench F_MH gap requires either (a) orchestration-stage multi-round refinement matching the long-chain structure (Q3 IterB ReAct), or (b) backbone upgrade (Backbone Matrix §5.1.10: Gemini-3-flash already narrows the F_MH gap meaningfully). Both paths are now empirically validated. The §5.5 Q3 IterC mechanism-class finding confirms that not all orchestration mechanisms transfer to EverMemBench F_MH equally (parallel decomposition vs sequential refinement). The §5.5.2 Q3 IterB ReAct result (D74) goes further: on the strongest backbone (Gemini-3-flash bare), multi-round retrieve-reason loop delivers **+2.01 pp clean F_MH lift (8.03% from 6.02% bare baseline)** — exceeding the Wave A/B/C single-stage retrieval ceiling of 7.25 pp by +0.78 pp standalone. The paradox is therefore refined rather than dissolved: EverMemBench F_MH is still largely a structural property of very long conversation chains × strict scoring, but MAS orchestration adds ~+2 pp on top of the strongest backbone above the retrieval ceiling — closing the gap is no longer purely structural, it now has both a backbone path and an orchestration path.

5.5 Q3 orchestration — IterC Self-Ask F_HL +35.84 pp, IterB ReAct ceiling break, and mechanism-class distinction

Q3 explores orchestration-stage mechanisms above the retrieval ceiling identified in Wave C (§5.1.9). Two deliverables are reported in this revision: **§5.5.1 Q3 IterC** (parallel decomposition via Self-Ask, F_HL +35.84 pp on the 5-batch protocol) and **§5.5.2 Q3 IterB** (sequential refinement via ReAct, F_MH retrieval-stage ceiling break on the strongest backbone). §5.5.3 records the mechanism-class distinction that emerged from the IterC/IterB contrast. §5.5.4 reports the composability projection for IterB stacked on top of Wave A/B/C retrieval-stage mechanisms (pending Q1 validation).

5.5.1 Q3 IterC — Self-Ask parallel decomposition (F_HL +35.84 pp)

Q3 IterC implements a Self-Ask-style sub-query loop: the generation model decomposes the query into sub-questions, each sub-question is retrieved separately, and a final synthesis step composes the answer.

Metric	Q3 IterC (5-batch)	Phase H v2 baseline	Δ
F_HL (high-level)	**58.52%**	22.68%	**+35.84 pp** PASS
F_MH (multi-hop)	~ baseline	3.21%	**−0.40 pp** (no-lift)
Overall	mixed	51.68%	—
Cost / latency	$0.0015/q + 2× LLM call	baseline	added cost

The F_HL +35.84 pp lift is the **largest single-mechanism F_HL improvement** measured in nox-mem's evaluation history. F_HL (high-level synthesis questions) benefits from parallel sub-query decomposition because the synthesis target itself is a composition over independent sub-facts — exactly the mechanism Self-Ask was designed for. The F_MH no-lift (−0.40 pp) is also informative: it forced the mechanism-class distinction documented in §5.5.3.

The Q3 IterC verdict: **ship opt-in** (`NOX_Q3_ITERC_ENABLED=1`) for F_HL-heavy workloads. NOT default-enabled (added cost + F_MH no-lift). The mechanism-class distinction reframed Q3 IterB ReAct as the leading EverMemBench F_MH candidate, which was then validated in §5.5.2.

5.5.2 Q3 IterB ReAct — F_MH retrieval-stage ceiling break on best backbone

IterB ReAct lifts F_MH **+2.01 pp** on Gemini-3-flash (95% CI [1.06, 9.39], overlapping baseline — directional, not significant). Detail in the anonymized supplementary material (§S5.5.2.)

5.5.3 Mechanism-class distinction — parallel decomposition vs sequential refinement

Self-Ask (parallel decomposition) and ReAct (sequential refinement) are different mechanism classes and do not substitute for each other. Detail in the anonymized supplementary material (§S5.5.3.)

5.5.4 Wave 2 closure — empirical composability matrix (detail in supplement) The full per-backbone × per-knob matrix (5-batch CLEAN, n=3,121) is in the anonymized supplementary material (§S5.5.4.) Headline: the 3-knob sum on Gemini-3-flash (KG −0.01 pp + AC +0.81 pp + MQ +1.21 pp) = **+2.01 pp aggregate = 24%** of the D74 pessimistic projection of +8.43 pp, which superseded that projection.

5.5.5 Wave 2 — Single-stage knob backbone-portability (D75) Re-running the three principal Lab Q1 retrieval knobs on Gemini-3-flash gives the transfer rates that define the finding:

knob	gpt-4.1-mini	Gemini-3-flash	transfer
KG path	+2.81 pp	**−0.01 pp**	**0%**
AC threshold=5	+2.01 pp	**+0.81 pp**	40%
MQ standalone	+3.61 pp	**+1.21 pp**	34%
3-knob sum	**+8.43 pp**	**+2.01 pp**	**24%**

Generalization principle. Any retrieval-stage lift of the form "knob X delivers +N pp on backbone Y" is **backbone-conditional**; cross-backbone generalization requires explicit re-measurement. Setup, mechanism interpretation and CIs: supplement §S5.5.5.

5.5.6 Wave 2 — MQ multi-axis backbone-conditional behavior MQ's F_MH lift attenuates across backbones (+3.61 → +1.21 pp) while its **MA composite flips sign** (−1.38 → +0.12 pp) and MA_U gains **+3.10 pp**, the strongest MA gain in Wave 2. Per-knob evaluation on a single metric axis can therefore hide compensating effects on orthogonal dimensions. Full axis table: supplement §S5.5.6.

5.5.7 Architectural composability vs mechanism composability The IterB adapter contains explicit guards at three locations in `eval/evermembench/adapter_nox_mem.py` that short-circuit the Wave A knobs when IterB is active — so IterB + Wave C was **architecturally** non-composable, independently of whether the mechanisms would compose. D74's projection implicitly assumed architectural composability; the code evidence shows that assumption was false. Code excerpts and design rationale: supplement §S5.5.7.

5.5.8 Wave 2 Capstone — D76 infrastructure abort (INDETERMINATE) The IterB + KG + rerank composability test was dispatched on the production VPS on 2026-05-31 and aborted after 48 h of sustained CPU steal, measured at **51–97%** while the ONNX cross-encoder rerank was active. The outcome is **INDETERMINATE — the hypothesis was not testable in that environment, which is neither a negative result nor a scientific failure**; it remains scientifically open, and completing it requires a plan with

a guaranteed CPU SLO. Failure timeline, attempted mitigations and the preserved branch: supplement §S5.5.8.

5.6 Cross-bench validation — LongMemEval[78] (n=300)

Config: Phase D production config (FTS5 + Gemini-embedding-001 + RRF, rerank OFF, top_k=20), GPT-4.1-mini backbone, Gemini-2.5-flash judge, oracle session retrieval, stratified n=300 queries.

Metric	Score
nDCG@10 (oracle retrieval)	**1.0000** (Wilson 95% lower bound 0.9872)
Recall@10	1.0000
Task accuracy (n=201 judged)	**68.16%** (Wilson 95% CI [0.6143, 0.7421])

Per-category breakdown — fingerprint consistency with EverMemBench:

Category	LongMemEval score	Strength	Matches EverMemBench pattern
single-session-assistant	87.10%	STRONG	PASS matches F_SH WIN
single-session-user	86.67%	STRONG	PASS matches F_SH WIN
knowledge-update	82.05%	STRONG	PASS matches MA_U WIN
abstention	82.61%	STRONG	(no EverMemBench equiv — unique strength)
multi-session	55.81%	moderate	PASS matches F_MH gap
temporal-reasoning	54.76%	moderate	PASS matches F_TP gap
single-session-preference	31.25% (n=16, wide CI)	weak	(preference handling weak)

The per-category fingerprint is **identical** to EverMemBench Phase D + H v2 results: strong on single-context factual + abstention + knowledge update; moderate on multi-session reasoning + temporal sequencing; weak on preference handling. This cross-bench consistency (two orthogonal benchmark distributions, different eval sets, different judges) confirms that nox-mem's strengths and weaknesses are **structural properties** of the retrieval architecture, not benchmark-specific tuning artifacts.

The Unicode-aware FTS5 sanitize fix is validated cross-bench: nDCG@10 improved from 0.9126 (Q2 baseline, pre-fix) to 1.0000 (+9.6 pp, oracle ceiling).

[78] Wu, Wang, Yu, Zhang, Chang & Yu, *LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory*, ICLR 2025. arXiv:2410.10813. The cross-bench validation set of §5.6.

This +9.6 pp delta is consistent in magnitude with the Q2 LongMemEval prefix measurement, confirming the fix's impact is not corpus-specific.

Caveat: oracle session retrieval is an upper-ceiling measurement. Comparison to gbrain (97.6% nDCG@10 on LongMemEval-S) requires the `s_cleaned` non-oracle follow-up (Lab Q1 priority, not yet run). The win claim is on **per-category task accuracy and abstention handling**, not on the oracle nDCG@10.

5.7 Operational characteristics — latency, cost, and footprint (self-hosted)

The canonical production boost stack (`section_boost × source_type_boost (Hard Mutex, query_entity_count <= 2) × salience v2 additive`) has been deployed since 2026-05-21 via systemd environment drop-in. Beyond accuracy, the production deployment establishes a unique competitive position on three operational dimensions:

5.7.1 Latency — sub-10 ms KG path

Path	p50	p95	p99	Notes
KG path (entity-walk)	**2.9 ms**	5.7 ms	—	SQL + regex over `kg_relations`, no LLM call; re-validated 2026-06-15, n=10 (the original run measured 2.5 ms / ~7 ms / ~14 ms)
Hybrid search (FTS5 + dense + RRF, no rerank)	~940 ms	~2,342 ms	~2,523 ms	Gemini-embedding-001 query dominates (~800 ms)
Hybrid + cross-encoder rerank (MiniLM)	+3,700 ms p50	—	—	Opt-in, exploratory mode

The KG path is in the sub-10 ms class at 2.9 ms p50 (re-validated 2026-06-15; the original run measured 2.5 ms — see the table above). Among the systems compared here, none reports retrieval latency in this band — but the comparison is not like-for-like and we do not treat it as a measured contrast: Zep's <100 ms p50 is a vendor-published figure, not independently verified by us (§7.1), and is therefore never used as a measured operand; the 100–500 ms

range cited for Mem0 Cloud and MemOS multi-service deployments comes from their own deployment documentation, not from our harness. What is ours and measured is the mechanism: nox-mem's single-process embedded architecture (better-sqlite3 + sqlite-vec in-process) eliminates network and IPC overhead entirely. **Re-validated 2026-06-15** on the 70.7k-chunk production corpus: the KG path (`/api/kg/path`, real entity pair, n=10) measured p50 = 2.9 ms / p95 = 5.7 ms, confirming the sub-10 ms class; the Gemini-embedding-dominated standard hybrid path measured p50 = 653 ms / p95 = 706 ms (n=20) — up from 529 ms as the corpus grew 69k → 70.7k chunks and reflecting Gemini API round-trip variance, which reinforces (rather than weakens) the case for the local KG path on latency-sensitive workloads.

5.7.2 Cost — $0/query KG path; hybrid vs managed SaaS is list-price, not like-for-like

Component	nox-mem	Mem0 Cloud (published pricing)	Ratio
Retrieval API cost (KG path)	**$0.00**	$0.001/query (est. embedding + retrieval)	**effectively free**
Retrieval API cost (hybrid w/ Gemini embedding)	$0.0000015/query	~$0.001/query (est.)	**~667× cheaper**
Ingest API cost (per chunk)	$0.00 (local)	varies	n/a
Total cost per 1M queries (hybrid)	$1.50	$1,000 (est.)	~667×

The KG path achieves **$0 per query** because the entity-walk uses only local SQL + regex with no LLM call (re-confirmed 2026-06-15). The hybrid path costs **$0.0000015/query** (gemini-embedding-001 at $0.15/1M input tokens — a Feb-2026 increase from $0.13/1M — × ~10 tokens/query). Against an *estimated* Mem0 Cloud per-query rate of ~$0.001 this is **~667× cheaper** (revised down from the 769× figure as Gemini embedding pricing rose). We lead with the unconditional claim — **$0/query KG path and zero marginal retrieval cost** — and treat the multiplier as secondary: Mem0 is sold as a subscription (free 1K calls/month, then $19–$249/month) with no published per-call overage, so the denominator is an explicit modeling assumption, not a quoted price.

5.7.3 Footprint — 399 MB RSS, single-process, self-hosted

Scaling	Idle RSS	10× concurrent	Notes
nox-mem-api process	**399 MB**	+15 MB (= ~414 MB)	better-sqlite3 + sqlite-vec single-process
Scaling pattern	flat	quasi-flat	No per-request memory blow-up

Self-hosted single-process means no multi-container orchestration, no Postgres/Redis/Chroma sidecars, no per-tenant container overhead. The 399 MB idle footprint runs on a $5/month VPS tier. Mem0 / Zep / Letta canonical deployments require >=3 services (API + DB + vector store) with combined RSS typically in the 1.5–3 GB range.

5.7.4 Observability layer The F10 observability layer (Phase A: `/observability/health.html`; Phase B: `/observability/evals.html`) renders the full G3→G10d ablation trajectory in real time over Chart.js, with gate annotations for D43, D48, D51, D67, D68, and D69. Three rollback paths are documented (conditional layer only, full mutex, drop-in removal), each executable in under five minutes.

5.8 Methodology, 5-batch protocol, and honest caveats

5.8.1 5-batch + 95% CI canonical methodology All EverMemBench claims in §5.1.5–§5.1.10 use the **5-batch canonical protocol**:

- **5-batch set:** batches 004, 005, 010, 011, 016 (n ~ 120–250 queries each, total n=3,121)
- **Aggregate metric:** mean across 5 batches per category
- **CI:** 95% confidence interval via t-distribution (n=5), reported as [lower, upper]
- **Claim threshold:** improvement claimed only when CI lower bound exceeds baseline mean

Single-batch gates were the prior protocol; they are now explicitly deprecated for any ship/reject decision.

5.8.2 Why single-batch gates overstate effects 3–6× The risk of single-batch measurement is concrete: Phase G batch 004 reported F_MH +8.00 pp (labelled "breakthrough"); the 5-batch reality was +1.61 pp — a 5× overstatement. Phase H v2 batch 004 reported +11.60 pp overall vs MemOS; the 5-batch reality was +9.13 pp (1.27× overstatement, still a win but a different narrative). The root cause is per-batch variance in EverMemBench: F_MH sigma ~ 2.3 pp, F_HL sigma ~ 5 pp, MA_C/P/U sigma ~ 3 pp — any single-batch Δ below ~2sigma is noise floor. Batch 004 specifically was a +1.40sigma to +1.70sigma upper-tail outlier across Phase G and Phase H runs; without the 5-batch protocol both would have been overclaimed in print. Additional single-batch failure mode: MA dimension regressions were **invisible** in Phase G batch 004 because batch 004 already had the lowest MA performance of the five batches (selection bias), hiding the −3 to −4 pp MA cost entirely.

Overstatement rates observed:

Phase	Metric	Single-batch	5-batch	Overstatement
Phase G	F_MH	+8.00 pp	+1.61 pp	5×
Phase G	F_TP	+11.67 pp	+2.00 pp	5.8×
Phase H v2	Overall	+11.60 pp	+9.13 pp	1.27×
Lab Q1 #4	F_MH	+6.78 pp (batch 004)	+2.81 pp	2.4×

5.8.3 MA dimension is mandatory in every eval report Memory Awareness (MA_C, MA_P, MA_U) is a **silent killer dimension**: Phase G batch 004 gate completely missed MA regression because MA was not measured in the initial single-batch run. Any retrieval change that involves reranking, query routing, or context modification must audit all three MA sub-dimensions, not just F_* and overall accuracy. MA regression indicates the change is damaging the system's ability to maintain user profile knowledge — the core differentiator of nox-mem vs retrieval-only systems.

5.8.4 Search error rate monitoring Concurrent agent operations during Lab Q1 benchmarking caused a batch contamination incident (batch 010): a concurrent agent re-installed its adapter mid-run, contaminating results. Recovery via merged adapter pattern. Lesson: shared adapter install paths on VPS are a race condition; sequential dispatch and 0/n search-error-per-batch monitoring are mandatory before accepting 5-batch results.

5.8.5 Honest scope of EverMemBench, LoCoMo, and classical-QA claims

- **EverMemBench Phase D headline (+2.95 pp vs MemOS Gemini)** is a modest win; the structural differentiator is the Memory Awareness composite, consistently strong across all backbones.
- **EverMemBench Phase H v2 headline (+9.13 pp vs MemOS GPT-4.1-mini)** is real and CI-verified, but the absolute score (51.68%) is not high — MemOS itself is only 42.55%. GPT-4.1-mini is the only tested backbone where all memory systems gain vs Full Context.
- **EverMemBench Backbone Matrix (Gemini-3-flash): +20.73 pp Overall / +32.74 pp MA composite** is the strongest cross-system claim in the paper. The lift is the multiplicative interaction of nox-mem's V10 retrieval stack and frontier-tier reasoning, not exclusively backbone-driven (§5.1.10).
- **MuSiQue-Ans dev answer F1 58.62%** (§5.2.1) and **HotPotQA distractor dev answer F1 73.37%** (§5.2.2) sit above the specialized readers these datasets are conventionally compared against (IRCoT, EX(SA), DPR+FiD) and roughly 10–12 points below current published SOTA (Beam Retrieval), without specialized fine-tuning. This bounds the architecture's multi-hop reasoning as competent rather than state-of-the-art, and is the sense in which §5.4 uses it.

- **LoCoMo retrieval@10 strict 74.52%** (§5.3) above Mem0 SOTA F1 66.88% is the retrieval ceiling on LoCoMo; the F1 push of 51.85% is rank-5 (above Zep / LangMem, below Mem0 SOTA 66.88%) due to a verbosity gap (§5.3.2). Path to >=55% requires orchestration (§5.5).
- **KG path, MAP, MQ, adaptive classifier, and Q3 IterC** are opt-in features, not defaults. Each addresses a known structural gap; combined effects measured in Wave B/C (§5.1.9).
- The Unicode-aware FTS5 sanitize fix is a prerequisite for all scores reported here; pre-fix Q2 numbers (nDCG@10 0.9126 LongMemEval) would have been reported as lower and should not be compared directly.
- **Wave 2 NO-REPLICATE findings (D75, §5.5.5):** the Lab Q1 single-stage knob lifts in §5.1.8 were measured on gpt-4.1-mini and are valid for that backbone. They do NOT transfer reliably to Gemini-3-flash (transfer rate ~0–40%). Any claim of "Wave A knob X delivers +N pp F_MH" must specify the backbone. The §5.5.4 composability matrix replaces the D74 projection with measured numbers; the original projection table is superseded and should not be cited.
- **IterB composability projection from D74 (IterB + Wave C → ~12.07% F_MH) is superseded.** The projection assumed both backbone-portability (refuted by D75) and architectural composability (refuted by adapter guard discovery in §5.5.7). The corrected empirical upper bound for measured+plausible IterB composability on Gemini-3-flash is ~8–9% F_MH (see §5.5.4 corrected table).
- **D76 capstone (§5.5.8):** the IterB + Wave C triple composability outcome is INDETERMINATE due to infrastructure abort. This is not a negative scientific result — it is an untested hypothesis. Batch 004 (n=49) is preserved but not 5-batch valid.
- **Limitations to flag (§5.8.6):** GPT-5 / Claude backbone columns are blocked by API access; the Zep <100 ms p50 claim is unverified by independent runs; the EverMemBench F_MH absolute number (3–7%) is not directly comparable to multi-hop reasoning gains on MuSiQue/HotPotQA — see §5.4 for the mechanism distinction.

5.8.6 Honest limitations and open work

- **EverMemBench F_MH absolute (3–7%) gap vs MemOS Table 4 (18.88%)** remains, and the §5.4 reframing argues it is not primarily a multi-hop reasoning failure, attributing it principally to the task setup — with the difficulty of the track visible same-metric in the 18.88% that the best published system reaches on it, rather than inferred from the classical benchmarks. Closing it requires either Q3 IterB ReAct (multi-round refinement on long conversation chains, §5.5.2) or backbone upgrade (Backbone Matrix §5.1.10 shows the gap narrows with Gemini-3-flash). Retrieval-stage knobs cap at ~+7.25 pp F_MH (D69 Wave C ceiling §5.1.9) and show low backbone-portability to Gemini-3-flash (D75 §5.5.5).
- **IterB composability with Wave A/B/C knobs on Gemini-3-flash**

is an open question. The D76 capstone (§5.5.8) was infrastructure-aborted before producing valid 5-batch data. The composability matrix in §5.5.4 documents this gap honestly. Completing the capstone requires dedicated CPU infrastructure.

- **LoCoMo F1 vs Mem0 SOTA 66.88%** remains open at rank-5 (51.85%). Wave C ceiling analysis (§5.1.9) indicates retrieval-stage knobs cannot close this gap; composition orchestration (Q3, §5.5) is the open path.
- **Zep <100 ms p50 claim** is published in marketing but not independently verified. nox-mem KG path p50 = 2.9 ms (§5.7) is measured on production VPS with the harness instrumented end-to-end. Comparison is fair only when both are measured under matched conditions.
- **GPT-5 / Claude columns** are blocked by API key constraints in the current eval setup. Backbone Matrix is currently three-cell (Gemini-2.5-flash, GPT-4.1-mini, Gemini-3-flash); GPT-5 and Claude entries are in the runway for Q3+ if access opens.
- **Wave A knob backbone-portability to other backbones** beyond Gemini-3-flash is unverified. The D75 ~30–40% transfer rate pattern is based on three knobs on one backbone pair. Additional backbone pairs (Claude Sonnet 4.6, GPT-5, Gemini 4) require independent re-baseline before composability projections can be made.
- **EverMind-AI / EverMemBench reference baselines** rely on MemOS Table 4 published numbers (arxiv:2602.01313); we have not re-run MemOS internally on the canonical 5-batch subset, only validated that the 5-batch sampling preserves the per-category distribution of the published numbers.

6. Q4 COMPARISON — Cross-System Benchmarking (Pre-registered)

6.1 Methodology summary

§6 covers the cross-system comparison between nox-mem and five competing persistent memory systems for AI agents. The complete execution plan is documented in `specs/2026-05-23-Q4-comparison-execution-plan.md` (pre-registered 2026-05-23, prior to the 2026-05-24 run). The comparison principles (§6.5), anti-cherry-pick guarantees (§6.6), and formal pre-registration (§6.7) are locked in this section before execution; only the tables in §6.2/§6.3/§6.4 receive numbers after the run. The objective is to satisfy the D43 approval gate — nox-mem in top-3 on >=2 of the 4 key metrics (nDCG@10, R@10, MRR, latency) — unlocking GTM Phase 2.

6.2 Competitors

The five competitors were selected by prioritizing GitHub stars, recent commit activity, and functional overlap with the nox-mem scope. Versions are locked

prior to execution for reproducibility.

System	Repo	Install path	Version pinned	Default config
Mem0	`mem0ai/mem0`	`pip install mem0ai==0.1.114`	`0.1.114`	OpenAI text-embedding-3-small 1536d + faiss (Chroma swapped to avoid telemetry thread-leak — §6.3.1)
Zep	`getzep/zep`	Docker compose (zep + postgres)	`0.27.2` — run 2026-09-10 on a host with a real Docker daemon, **not** in the 2026-06-15 canonical (§6.3.4)	Local self-host mode; `text-embedding-3-small` 1536d, **no reranker**
Letta (ex-MemGPT)	`letta-ai/letta`	`pip install letta` (+ `click<8.2`)	`0.6.6`	SQLite backend; agent-OS retrieval
agentmemory	`rohitg00/agentmemory`	daemon (REST :3111)	`as-of 2026-06-15`	union store + namespace filter
EverMind-AI	EverOS (PyPI)	`pip install everos`	`1.3.1` — run 2026-09-10, **not** in the 2026-06-15 canonical (§6.3.3)	local-first library; `gemini-embedding-001` 3072d + `Qwen/Qwen3-Reranker-4B` rerank (**mandatory**, §6.3.3)

Each system runs with its publicly documented default configuration (principle 3 of §6.5): no competitor is adversarially tuned to underperform.

6.3 Per-system per-dataset results

Canonical cross-system × cross-dataset table. K cutoff fixed at 10 across all systems; latency measured externally (wall clock around adapter call); cost derived from per-system logs (API calls × published pricing).

Preliminary smoke (2026-05-24) — superseded. An initial 20-query smoke on an eval-isolated DB validated the harness end-to-end (nox-mem, full corpus: nDCG@10=0.6380, gold-hit 13/20; Mem0 at a 500-chunk cost-control cap, not directly comparable). Its only role was pipeline validation; it is **superseded** by the canonical run (below) and rc4 (§6.3.2), and the capped-corpus "concentration vs coverage" reading no longer bears on the reported results. The per-dataset breakdown below is from the **canonical run (2026-06-15)**; rc4 (§6.3.2) extends it to the full n=2,482 set.

Canonical run — 2026-06-15 (dedicated RunPod pod, n=100/dataset, same-namespace fair). After the 2026-05-24 infrastructure abort (§7.1 L5), the canonical cross-system run was executed on a dedicated pod on 2026-06-15. **Two of the five competitors produced head-to-head quality numbers** (Mem0, agentmemory; nox-mem is the system under test, not a competitor); **the other three are documented deployment non-runs** with explicit reasons (Zep, Letta, EverMind-AI — §6.3.1). Protocol: n=100 queries per dataset, k=10; retrieval quality (nDCG@10 / recall@10 / MRR) under each system's native default embedding provider; **same-namespace fair re-query** — the union store is queried at k=30 and filtered to the active dataset namespace before re-scoring the top-10, removing the cross-dataset distractor confound that inflated an early union-store reading (14.6% of nox-mem's raw top-10 were off-dataset chunks). Per-system latency percentiles were not captured uniformly in this run; nox-mem operational latency is reported independently in §5.7 (KG path 2.9 ms p50, hybrid 653 ms p50, re-measured 2026-06-15).

LongMemEval n=100 (canonical):

System	nDCG@10	R@10	MRR	Cost/query	Coverage / config
nox-mem	**0.5234**	0.6535	**0.5494**	**$0 (local)**	100%; hybrid FTS5 + Gemini-3072d + RRF
Mem0	0.4764	0.6372	0.5107	subscription + OpenAI embed	100%; faiss backend, OpenAI text-embedding-3-small 1536d
agentmemory	0.2803	n/c	n/c	$0 (local)	daemon REST; weak retriever
Zep	[not in this run - Docker impossible on the unprivileged pod (§6.3.1); measured 2026-09-10 over the full n=2,482 set on a Docker-capable host, §6.3.4]	—	—	—	—

System	nDCG@10	R@10	MRR	Cost/query	Coverage / config
Letta	[GAP - see §6.3.1: agent-OS latency ~16 min/query; impractical for n=100]	—	—	—	—
EverMind-AI	[not in this run - barrier of §6.3.1 lifted later; measured 2026-09-10 over the full n=2,482 set, §6.3.3]	—	—	—	—

LoCoMo n=100 (canonical):

System	nDCG@10	R@10	MRR	Cost/query	Coverage / config
Mem0	**0.4686**	0.6171	0.4656	subscription + OpenAI embed	~95% (thread-leak ingest loss); faiss, 1536d
nox-mem	0.4263	0.5504	0.4464	**$0 (local)**	100%; hybrid FTS5 + Gemini-3072d + RRF
agentmemory	0.1587	n/c	n/c	$0 (local)	daemon REST; weak retriever
Zep / Letta / EverMind-AI	[GAP of this run - see §6.3.1; Zep and EverMind-AI have since produced numbers (§6.3.3, §6.3.4)]	—	—	—	—

Split result — honest reading (mandatory per §6.6). Each top-tier system wins one benchmark: **nox-mem wins LongMemEval** (+0.047 nDCG@10, +0.039 MRR), **Mem0 wins LoCoMo** (+0.042 nDCG@10). agentmemory is a

distant third on both, reflecting a weaker retriever. This is a genuine **split, not a dominance claim**: nox-mem is competitive with the market leader on retrieval quality *while* delivering the operational profile of §6.8 (single SQLite file, $0/query KG path, no service stack). The nox-mem LoCoMo figure rose from 0.4173 (raw union store) to 0.4263 after the namespace fix — Mem0 still wins, confirming the LoCoMo result is a real retrieval gap, **not** merely a confound artifact. Per §6.6, both datasets are reported side by side; we do not cherry-pick the one nox-mem wins.

Caveats (per-system composition, declared per §6.5). (a) **Embedding providers differ by native default** — nox-mem Gemini 3072d, Mem0 OpenAI 1536d, agentmemory its own default; this is the "as-configured" fair-comparison stance (§6.5, principle 5); the controlled-embedding experiment that equalizes this confound is reported in §6.3.2 (rc4, all-Gemini) and inverts the LoCoMo result. (b) **Store composition differs**: nox-mem and agentmemory were queried from a union store with namespace filtering; Mem0 used single-namespace stores (its LoCoMo store ingest leaked threads and lost ~5% of vectors — §6.3.1). (c) **Coverage**: nox-mem 100% both datasets; Mem0 100% LongMemEval / ~95% LoCoMo; agentmemory full. These are reported, not hidden, consistent with §6.6.

6.3.1 Documented gaps — one system that did not run Per §6.6, systems that fail setup receive an explicit reason rather than silent omission. The 2026-06-15 run hit three:

- **Zep** — requires a Docker stack (Zep + Postgres). Docker is **impossible on the unprivileged benchmark pod**: `dockerd` fails on `iptables ... Permission denied` (no `NET_ADMIN`), and every documented workaround (`--iptables=false --bridge=none`, `--storage-driver=vfs`) fails on kernel-namespace syscalls blocked by the pod seccomp profile. This is a privilege constraint, not a configuration error; Zep requires a host with real Docker (privileged VM or its hosted Cloud).

  Zep is a non-run of the 2026-06-15 canonical run specifically — it did run earlier, and the artifact is in this repository. `eval/q4-comparison/output/zep.json` (2026-05-25, `zep-python==1.5.0` + `ghcr.io/getzep/zep:0.27.2` OSS via Docker) holds **20 queries, 20 with results, 0 errors** — from the superseded 500-chunk-cap smoke (§6.3, "Preliminary smoke"), on a host that did have Docker, not on the benchmark pod. We state this because the repository is public and linked from this paper: a reader who opens `output/` finds a Zep run, and "Zep did not run" without this note would read as a contradiction. The smoke is not comparable to the canonical run (different corpus cap, n=20 vs n=100/dataset, no same-namespace re-query), so its numbers are not promoted into §6.3 — but its existence is disclosed rather than left for the reader to discover. **A comparable Zep run now exists** (2026-09-10, full n=2,482, §6.3.4) and lives at a different path, `output-2026-09-10/`;

`output/zep.json` is left where it is because this paper cites it.

- **Letta (ex-MemGPT)** — installs and runs natively (CLI unblocked via `click<8.2`, server starts without Docker), but is an **agent-OS**: retrieval routes through a full LLM agent turn (`archival_memory_search` tool call), measured at **~16 min/query** — n=100 × 2 datasets would take days. Ingest also degrades catastrophically (stalls at ~94% after ~12 h on a sequential archival-memory insert). Letta *validates* (it runs) but is impractical for a 100-query benchmark in this configuration. **Zep was a gap of this list and no longer is.** The barrier was the *host*, not the software: `dockerd` cannot start on the unprivileged pod. On a host with a real Docker daemon and a paid OpenAI key, the same OSS stack ran the **full n=2,482 canonical set on 2026-09-10 with zero errors** — reported in §6.3.4. The bullet above is kept, dated, because the deployability finding of §6.8 was made against the 2026-06-15 environment and that finding (a paid LLM key is mandatory, verified in source) is unchanged by the run.

EverMind-AI / EverOS was a gap of this list and no longer is. At the 2026-06-15 canonical run it ran only as an HTTP service and required two third-party credentials not available to that run (OpenRouter for LLM extraction + DeepInfra for embedding/rerank), plus authorization for an external-repo `pip install -e`; it was out of scope for that time-box. That barrier was lifted upstream: `pip install everos` now resolves to a local-first library (v1.3.1) with no mandatory OpenRouter credential. **It was run on 2026-09-10 over the full n=2,482 set and produced a number, reported with its confounds in §6.3.3.** It is kept in this subsection, rather than deleted from it, because the deployability finding of §6.8 was made against the version that existed on 2026-06-15 and silently removing the record would misdate that comparison.

What is measured of EverOS, and what is not (2026-09-10). Credentials were obtained after the canonical run, and the corpus was ingested into a live EverOS service: **6,822 documents retained**, measured on the searchable index after the final sync, from 6,830 offered lines carrying 6,822 distinct ids. Four of the 6,830 attempts failed, all four with `DuplicateDocumentError` on the eight ids that name two distinct documents each; no id is absent from the index and none of the four is the gold document of any query (predicate: `intersect(corpus_ids - indexed_ids, union(gold_chunk_ids)) = empty`). *"Zero failures"* and *"four failures, none reachable"* are different states and this is the second — the predicate is stated so the difference is not read away.

One configuration fact of that service bounds any future retrieval number and is recorded here because it was measured, not predicted: EverOS reranks with a `rerank_n` of **50** while exposing a parameter named `top_k_cap = 100`, and every query in the sweep returns exactly **50 hits** under `method=hybrid` — **saturation at the ceiling on 100% of queries**, never at the value the parameter's name advertises. Any nDCG@10 from that service is therefore computed over a candidate set truncated at 50, and the name of the larger parameter overstates

the limit that binds.

That sweep has since closed, and its number is reported in §6.3.3.

LightRAG and HippoRAG2 are **not §6 competitors** — they are §5 KG/RAG references whose LLM-per-chunk graph-build cost (LightRAG warns >6 h on default) places them outside the memory-system comparison; they are deferred as optional baselines (§7.2).

6.3.2 Embedding-matched variant (rc4 — all-Gemini, full eval) The §6.3 split is reported under each system's *native default* embedder (the "as-configured" stance, §6.5, principle 5). Because nox-mem defaults to Gemini 3072d and Mem0 to OpenAI 1536d, the most obvious confound on the split is the embedding provider itself. To probe it we ran an **embedding-matched** variant — **rc4** — in which both systems use the **same embedder**: `gemini-embedding-001` at 3072d (the model and dimensionality nox-mem runs in production), with Mem0 re-pointed to it via an external config patch (`lib/all_gemini_config`). This was **not** a zero-edit swap: the Mem0 adapter also required a one-time API-compatibility fix for the mem0 2.x `search`/`get_all` signatures (declared as confound (a) below). The variant also replaces the n=100 sample with the **full evaluation set (n=2,482: 1,982 LoCoMo + 500 LongMemEval)** over the full 6,822-chunk corpus, scored identically to §6.3 (binary-relevance nDCG@10, k=10, same gold encoding). The evaluation gold is fully covered — every gold chunk exists in the corpus on both datasets — but note this is *corpus/gold* coverage, **not** Mem0 *store* coverage (Mem0 dropped 4 of 6,822 chunks on ingest; confounds below). This is an embedding-**matched** comparison (same model + dimensionality), **not** an all-else-equal controlled experiment: four residual confounds are declared below, and a fourth — the embedding task-type asymmetry — is tested by a dedicated ablation and shown not to drive the result.

System (all-Gemini 3072d)	LongMemEval nDCG@10 (n=500)	LoCoMo nDCG@10 (n=1,982)	Overall (n=2,482)
nox-mem (hybrid FTS5 + Gemini + RRF)	**0.5255 ± 0.033**	**0.4952 ± 0.017**	**0.5013 ± 0.015**
Mem0 (Gemini embedder, Chroma)	0.4061 ± 0.036	0.4407 ± 0.017	0.4337 ± 0.016

Intervals are 95% confidence (mean per-query nDCG@10 ± 1.96 · SEM (standard error of the mean), binary relevance). The nox-mem / Mem0 intervals are **disjoint on both datasets and overall**, so the gap is not attributable to query-sampling noise (it remains subject to the four residual configuration confounds below; the one asymmetry that favored nox-mem — task type — was ablated and shown not to drive the result).

Result — the split does not survive embedding matching. With the embedding model and dimensionality equalized (and the four residual confounds below declared, a fifth — task type — ablated away), **nox-mem outperforms Mem0 on both benchmarks** (LongMemEval +0.119, LoCoMo +0.055 nDCG@10) and **all five represented query categories** (§6.4). The Mem0 LoCoMo win in §6.3 was therefore substantially an embedder effect — OpenAI 1536d on LoCoMo — rather than a retrieval-architecture advantage: once both systems embed with the same Gemini model, nox-mem's hybrid FTS5 + dense + RRF retrieval wins on LoCoMo as well.

Residual confounds — declared; this is embedding-matching, not a clean architecture isolation (per §6.6). Four factors remain that prevent attributing the inversion purely to the embedding; the last of them, **(e)**, is quantified below and runs *against* nox-mem (a further one — task-type asymmetry — was tested by ablation and neutralized; see below). **(a) Mem0 version — not recorded, and the failure mode is worse than drift.** The version Mem0 ran under in rc4 cannot be established from what was persisted. `output/rc4/mem0.json` carries `meta.version = "mem0ai==0.1.114"`, but that string is the adapter's declared `VERSION_PIN` — *intent* — not a runtime read: `validate()` does capture `mem0.__version__`, and its output was never written to the artifact. Table §6.2 and the artifact are therefore **one source, not two**, so their agreement corroborates nothing. Earlier revisions of this section asserted a `0.1.x → 2.0.10` drift; that number came from the pin, which was bumped to `2.0.10` **3 h 47 min after** the run's own `finished_at` (pin bump 2026-06-29T18:44:38Z; run finished 14:57:04Z) — it describes the repository after the run, not the run. Nor does the run's clean exit (`n_errors: 0` over n=2,482) discriminate: in v2.0.10 `search` and `get_all` are keyword-only with `**kwargs`, so the 0.1.x-shaped calls the adapter made at run time (`search(query=…, user_id=…, limit=k)`) would be **silently absorbed** rather than rejected; and the obvious discriminator — the length of the returned list — is destroyed by the adapter's own `items[:k]` slice (all 2,482 queries show exactly 10, by construction). **Stated as risk, not as claim:** *if* 2.0.x was installed, `user_id` and `limit` were dropped and Mem0 searched with `filters=None` (unfiltered) at `top_k=20`, which would make rc4's Mem0 column *invalid* rather than merely drifted. We do not assert that this happened; we record that the artifacts cannot exclude it. **Required of any future run:** persist `validate()`'s `mem0.__version__` into `meta`, so the recorded version is state rather than intent. **(b) Vector backend:** canonical Mem0 used a faiss store; rc4 Mem0 uses a fresh Chroma collection (required to avoid a dimension clash with the 1536d OpenAI run) — a backend change, not only an embedder change. **(c) Sample scope:** §6.3 sampled n=100/dataset; rc4 scores the full n=2,482 — a larger, differently-distributed query set, not a like-for-like re-score of the same 100. rc4 is therefore best read as *"same embedding model and dimensionality, Mem0 as installed on the rc4 pod (version unrecorded) and at its default backend"* → nox-mem leads — **not** a surgical architecture-only isolation. (The 4 chunks Mem0 dropped to transient 503s affect one query, whose gold chunk Mem0 fails to retrieve in top-

10 even when present — verified zero aggregate effect.) **(e) Corpus retention differed by adapter — and against the arm that won.** The offered corpus is 6,830 documents carrying 6,822 distinct ids: 8 pairs of documents with *different text* share an id. nox-mem's eval loader inserts with `INSERT OR IGNORE`, so it retained **6,822** and silently kept only the first document of each colliding pair; Mem0's Chroma collection retained all **6,830**. Measured 2026-09-10 on the run's own artifacts — `cache/rc4-nox-hybrid.db eval_chunks` = 6,822; `.mem0-chroma-rc4/chroma.sqlite3 embeddings` = 6,830 with 6,822 distinct `chunk_id`. Scoring is by id, so a collision cannot mis-score a retrieval; 10 of 2,482 queries (0.40%) carry a gold id in a colliding pair, and on those nox-mem held **one** candidate text where Mem0 held two. The direction matters more than the magnitude: this asymmetry disfavours nox-mem, which won anyway. Which document of a pair survives `INSERT OR IGNORE` is deterministic but arbitrary — it is file order, not a choice. One external datum bears on how to read this, from a **different run** (the 2026-09-10 EverOS ingest of §6.3.1, not rc4, and therefore not a fourth column here): offered the same corpus, EverOS also retained **6,822**, reaching that number by rejecting the duplicate id with an explicit error rather than ignoring it in silence. Two systems, two mechanisms, one count — which makes 6,822 the consequence of honouring id uniqueness rather than an idiosyncrasy of our loader. It does not make the rc4 asymmetry smaller: against Mem0's 6,830, the handicap stands as stated. The per-query outputs are not distributed; the run is reproducible from `eval/q4-comparison/runner_rc4.py` and `eval/q4-comparison/run_rc4_full.sh`.

Task-type ablation — confound (d) tested and neutralized. The one configuration asymmetry that *favored* nox-mem in rc4 was the embedding task type: nox-mem passes Gemini's `RETRIEVAL_DOCUMENT`/`RETRIEVAL_QUERY` task types (retrieval-optimized embeddings), while Mem0's embedder call does not. To isolate it we re-ran nox-mem with a **generic Gemini embedding** (`NOX_EMBED_GENERIC_TASKTYPE=1` — no task type, exactly how Mem0 calls the same model) against the **same** Mem0 baseline, over the full n=2,482 — a symmetric *"neither system sets a task type"* comparison holding confounds (a)–(c) constant. nox-mem's overall nDCG@10 falls only **0.5013 → 0.4979 (−0.34 pp)** and **still outperforms Mem0 (0.4337) on overall, both datasets (LoCoMo 0.4920 vs 0.4407; LongMemEval 0.5215 vs 0.4061), and all five categories**. The task-type asymmetry therefore contributes at most 0.34 pp and does **not** explain the §6.3 → §6.3.2 inversion: the win is architectural (hybrid FTS5 + dense + RRF fusion), not an embedding-mode artifact. (Rigor caveat: the re-ingested generic corpus reached 99.03% gold coverage — 23 of 2,370 distinct gold chunks absent vs 100% in the task-type run, a transient-ingest handicap that can only *lower* nox-mem's score; the win persists despite it.) The ablation is reproducible from `eval/q4-comparison/run_rc4_ablation.sh`.

LoCoMo claim taxonomy (orientation). "Who wins LoCoMo" depends entirely on metric and configuration; the paper reports four distinct, non-contradictory readings, tabulated here to prevent conflation:

§	Metric	Configuration	n	Result
§5.3.1	retrieval@10 (strict)	nox-mem standalone	—	nox-mem 74.52% (vs Mem0's *published* F1 66.88% — different metric, not head-to-head)
§5.3.2	answer F1	nox-mem standalone	—	nox-mem rank-5 (below Mem0; verbosity-constrained, §5.3.2)
§6.3	nDCG@10	native embedders (nox Gemini-3072d / Mem0 OpenAI-1536d)	100	**Mem0** 0.4686 vs nox-mem 0.4263
§6.3.2	nDCG@10	matched embedder (both Gemini-3072d)	2,482	**nox-mem** 0.4952 vs Mem0 0.4407

The §6.3 → §6.3.2 reversal is the embedding-matching effect (with the four residual confounds above; the task-type asymmetry was ablated and ruled out); the §5.3 retrieval-vs-F1 contrast is an orthogonal metric distinction, not a contradiction.

6.3.3 EverOS measured (2026-09-10) — a competitor that outperforms nox-mem, and the pipeline confound that qualifies it The EverMind-AI gap of §6.3.1 closed upstream. `pip install everos` resolves to a local-first library (v1.3.1) that needs no HTTP service and no mandatory OpenRouter credential, and it was run over **the same corpus and the same n = 2,482 query set as rc4** (§6.3.2), scored by the same aggregator under the same binary-relevance nDCG@10 at k = 10.

System	Overall nDCG@10 (n=2,482)	LoCoMo (n=1,982)	LongMemEval (n=500)	R@10	MRR	p50 latency
EverOS 1.3.1 (2026-09-10)	**0.6455**	**0.6585**	**0.5942**	0.7629	0.6403	1,592 ms
nox-mem (rc4, 2026-06-29)	0.5013	0.4952	0.5255	—	—	653 ms (§5.7)
Mem0 (rc4, 2026-06-29)	0.4337	0.4407	0.4061	—	—	not captured

EverOS outperforms nox-mem here, on both datasets — +0.163 LoCoMo, +0.069 LongMemEval, +0.144 overall — across 2,482 queries with zero errors. Per §6.6 the result that goes against us is reported in the same table as the ones that do not, and in the same revision that measured it.

The confound, and what we do not claim about it. The two pipelines are not the same shape, and the difference is the cross-encoder re-ranking stage[79]. EverOS **requires** a cross-encoder: `_require_search_providers()` in `everos/service/knowledge.py` raises `ProviderNotConfiguredError` before any search path when a reranker is absent, so `Qwen/Qwen3-Reranker-4B` is its minimum viable configuration, not a generous setting we chose for it. nox-mem's rc4 run has **no cross-encoder stage at all**: what its adapter calls rerank is RRF re-ordering plus a 1-hop KG-neighbourhood multiplier. This is a property of the systems compared rather than an experimenter's choice — and **its magnitude is not measured in this benchmark.** The only measurement we hold of a cross-encoder's contribution inside nox-mem is §5.1.7: **−0.96 pp overall**, +1.61 pp on multi-hop, −2.80 to −4.00 pp on Memory Awareness. That study used **MiniLM-L-6-v2, 22 M parameters** — roughly two orders of magnitude smaller than a 4 B reranker — on a different benchmark, under different metrics, through a different code path. It does not transfer, and we do not offer it as a rebuttal. We record it because it is the only evidence we hold on this term at all — and at a 180-fold parameter gap it barely serves as a floor. What it supports is the negative claim: **we do not know the magnitude**, and nothing in this paper licenses attributing the 0.144 gap to the reranker, in whole or in part. Settling the question means adding the stage to the nox-mem adapter and re-running the same corpus (§7.2) — not flipping a flag, because the stage does not exist in this harness.

Three further asymmetries, declared per §6.5. (a) *Run date* — rc4 ran 2026-06-29, EverOS 2026-09-10; corpus, query set, embedding model and dimensionality (`gemini-embedding-001`, 3072 d) are the same, the calendar is not. (b) *Operational axis* — EverOS's p50 is 2.4× the nox-mem hybrid path, and every EverOS query is paid on two providers (DeepInfra rerank + Gemini), against $0 on the nox-mem KG path and one embedding call on its hybrid path (§6.8); the quality column and the cost column point in opposite directions here, which is the whole reason §6.8 exists. (c) *Configuration provenance* — the artifact's `meta` records what was asked of the harness and nothing about model, dimensionality or reranker; the configuration named above was captured from the live process's environment before it exited. That omission is recorded as a debt, not reconstructed after the fact.

On what this number is, and is not, relative to the vendor's own.

[79] Nogueira & Cho, *Passage Re-ranking with BERT*, 2019. arXiv:1901.04085. The work that established the cross-encoder as a re-ranking stage over a first-stage candidate pool: the MS MARCO result cited in §7.2 F5, and the stage whose **presence or absence** is the confound of §6.3.3–§6.3.4 — EverOS requires one, nox-mem fuses by RRF[^rrf] without one, Zep has neither. Our own measurement of the stage (§5.1.7, −0.96 pp) used a 22 M distillation[^minilm], ~180× smaller than the 4 B model EverOS runs[^qwen3embed], so it is not carried over as a bound.

EverMind-AI reports EverMemOS[80] at 93.05% on LoCoMo and 83.00% on LongMemEval. Those are **LLM-judged answer accuracy**; ours is **retrieval nDCG@10** over a fixed corpus, so the two are not the same quantity and 0.6455 neither confirms nor contradicts them. Two structural points are worth stating anyway, because they cut in opposite directions. Against the vendor: their headline numbers are self-reported, and the benchmark on which several of them are obtained, EverMemBench[81], is authored by the same group — a reviewer of §6 should hold our own EverMemBench numbers (§5.1) to that identical standard, which is why §6.6 forbids us from reporting a benchmark we win without the one we lose. In the vendor's favour: the measurement above is, as far as we can establish, an **independent third-party run** of their system, and it is a good one — a competitor beating us on the axis we optimize, measured by us, published by us.

Artifact: `eval/q4-comparison/output-2026-09-10/_aggregate.json` (`everos==1.3.1`, n_queries 2,482, n_errors 0).

6.3.4 Zep measured (2026-09-10) — the last Docker gap closed, and the system places fourth The Zep gap of §6.3.1 was a **host** gap, not a software one: `dockerd` could not start on the unprivileged benchmark pod. Given a host with a real Docker daemon and a paid OpenAI key, the same OSS stack of the 2026-05-25 smoke (`zep-python==1.5.0` + `ghcr.io/getzep/zep:0.27.2`, Zep + Postgres/pgvector) ran the **full canonical set, n = 2,482, zero errors**, in 4 h 13 min.

System	Overall nDCG@10 (n=2,482)	LoCoMo (n=1,982)	LongMemEval (n=500)	R@10	MRR	p50 latency
EverOS 1.3.1 (2026-09-10)	0.6455	0.6585	0.5942	0.7629	0.6403	1,592 ms

[80] Hu, Gao, Zhou, Xu, Bai, Li, Zhang, Li, Zhang, Bing & Deng, *EverMemOS: A Self-Organizing Memory Operating System for Structured Long-Horizon Reasoning*, arXiv:2601.02163, 2026. Implementation: github.com/EverMind-AI (~5k stars, Apache 2.0). Publishes EverMemBench dataset and an EvoAgentBench-framed evolution loop. The only memory-OS peer in our taxonomy that publishes its own benchmark; §3.4 is the direct narrative counter to EvoAgent framing. Honest-comparison action item: run nox-mem on EverMemBench (queued as F4 in §7.2). This is also the system paper for what §6.3.3 measures — §6.3.3 reports a quality number **of** this system, and citing only its repository would leave the reader unable to check what was built. Its reported 93.05% LoCoMo / 83.00% LongMemEval are LLM-judged answer accuracy, a different quantity from the retrieval nDCG@10 of §6.3.3, and are not compared to it.

[81] Hu et al., *Evaluating Long-Horizon Memory for Multi-Party Collaborative Agents* (the EverMemBench paper), 2026. arXiv:2602.01313. Named in §1.5 as a setting nox-mem has not been measured on — an open gap, not a claimed capability. §5.1.5–§5.1.10 report nox-mem numbers **on** this benchmark and §6.3.3 reports a number **of** the system whose authors also wrote it; both facts are disclosed rather than left implicit, and the disclosure applies symmetrically to our own use of it.

System	Overall nDCG@10 (n=2,482)	LoCoMo (n=1,982)	LongMemEval (n=500)	R@10	MRR	p50 latency
nox-mem (rc4, 2026-06-29)	0.5013	0.4952	0.5255	—	—	653 ms (§5.7)
Zep 0.27.2 (2026-09-10)	**0.4546**	**0.4793**	**0.3567**	0.6108	0.4279	**6,002 ms**
Mem0 (rc4, 2026-06-29)	0.4337	0.4407	0.4061	—	—	not captured

Zep places **fourth of four** on overall nDCG@10, and its LongMemEval number (0.3567) is the lowest any system in this table records on either dataset — below its own LoCoMo by 0.123, the widest per-dataset spread here. Its p50 of 6,002 ms is **9.2×** nox-mem's 653 ms and **3.8×** EverOS's 1,592 ms; the p99 is 11,846 ms.

Why the latency, and why it is a property of the design rather than of our host. Zep has no cross-corpus index: retrieval is per *session*, so a query over this corpus fans out to **510 sessions** and the client merges the results. We ran that fan-out in parallel, at the highest worker count the server pool sustained; the number above is what the fastest configuration we could give it produced, not a throttled one. The comparison is therefore honest in the direction that hurts us least to state: a single-session Zep deployment would not pay this cost, and this corpus is not a single session.

What Zep does and does not do in the pipeline (per §6.3.3's confound). Zep embeds the query through OpenAI (`text-embedding-3-small`, 1,536 d) and **does not rerank** — no cross-encoder stage, confirmed in `zep-config.yaml` (Summarizer, Entity and Intent extractors all disabled). So the three-way shape is: EverOS reranks with a 4 B cross-encoder[82] and **cannot run without one** (§6.3.3), nox-mem fuses by RRF with a KG neighbourhood term and no cross-encoder, Zep does neither. We do not convert that ordering into an explanation of the scores; the magnitude of the pipeline effect is unmeasured here, exactly as declared in §6.3.3.

Declared, not corrected: 47 session-level failures. Across the run's 1,265,820 session sweeps (2,482 queries × 510 sessions, 0.0037%), 47 failed in 46

[82]Zhang, Li, Long, Zhang, Xie *et al.*, *Qwen3 Embedding: Advancing Text Embedding and Reranking Through Foundation Models*, 2025. arXiv:2506.05176. Named in §7.2 F5 as the current open-weight reranker candidate **for nox-mem's own stack, where it is not run**. It is not, however, absent from this study: EverOS reranks internally with a Qwen3 reranker, and §6.3.1 records the `rerank_n = 50` ceiling that this imposes on the candidate set measured there. The model appears here as a component of a *compared system*, never of ours.

distinct sessions: **35** client-side timeouts, **8** OpenAI 500s on `/v1/embeddings` after six retries, and **4** `Bad file descriptor` from our connection pool. **None is a Zep retrieval failure**, and none crossed the per-query abort threshold — a crossing would have become a query error, and `n_errors` closed at 0. Each removed one of 510 sessions from that query's candidate pool.

Artifact: `eval/q4-comparison/output-2026-09-10/_aggregate.json` (`system: zep`, n_queries 2,482, n_errors 0). **Caveat.** This is **not** `eval/q4-comparison/output/zep.json`, which is the superseded 2026-05-25 smoke (20 queries, no nDCG) cited elsewhere in this paper; the two are different runs and the older path is kept unmoved because it is cited.

6.4 Per-category breakdown

Per-query-category breakdown from the **rc4 embedding-matched run** (§6.3.2; all-Gemini 3072d, n=2,482 across both datasets). The canonical 2026-06-15 run reported only dataset-level metrics, so the breakdown is populated from rc4, where per-category gold labels — the datasets' *native* question types, mapped to six canonical buckets — are wired into the harness. Only nox-mem and Mem0 participate: agentmemory's server-side embedder cannot be re-pointed to Gemini, and the three §6.3.1 gaps likewise do not run under the matched embedder.

Category	n	nox-mem nDCG@10	Mem0 nDCG@10	Δ
single-hop	438	**0.3969**	0.3908	+0.006
multi-hop	454	**0.5481**	0.4699	+0.078
temporal	225	**0.3940**	0.2760	+0.118
adversarial	524	**0.4370**	0.2955	+0.142
open-domain	841	**0.5991**	0.5649	+0.034
numeric	n/a	n/a	n/a	—

nox-mem leads every represented category. The largest margins are on **adversarial** (+0.142) and **temporal** (+0.118) — the query types where naive semantic similarity (Mem0's Gemini-only path) is weakest and the FTS5 lexical channel in nox-mem's hybrid fusion contributes most; the narrowest is **single-hop** (+0.006), where a single strong dense match suffices and the hybrid adds little. This pattern is consistent with the §5 ablations attributing nox-mem's edge to multi-signal fusion rather than embedding quality alone, and survives the task-type ablation (§6.3.2): with a generic Gemini embedding, nox-mem still leads all five categories (adversarial 0.4573 vs 0.2955; multi-hop 0.5561 vs 0.4699; open-domain 0.5792 vs 0.5649; single-hop 0.3924 vs 0.3908; temporal 0.3765 vs 0.2760).

Bucket composition (declared per §6.6). The six buckets aggregate the two datasets' native labels, and two mappings warrant disclosure because they affect interpretation. The **adversarial** bucket (n=524) combines LoCoMo's 446 native adversarial queries with 78 LongMemEval `knowledge-update`

queries — a mapping flagged as *ambiguous* in the labeling audit (`eval/q4-comparison/docs/rc2-per-category-mapping.md`), since `knowledge-update` is not adversarial in LoCoMo's sense; the adversarial cell (nox-mem's largest margin, +0.142) should be read as "adversarial + knowledge-update", not pure adversarial. The `numeric` category receives `n/a`: neither dataset contributes at least 10 numeric-typed queries (open-domain is likewise LongMemEval-only), so per the n<10 rule those cells are not scored rather than extrapolated — avoiding the type of regression seen in §5.1.4 (temporal `n/a` in G10b due to a degenerate corpus gap). The category labeler is `eval/q4-comparison/lib/category_labeler.py`.

6.5 Fair-comparison principles

The comparison adheres to principles standardized by the published benchmarking literature (EverMemBench; BEIR, Thakur et al. 2021, arXiv:2104.08663; MTEB, Muennighoff et al. 2023, arXiv:2210.07316):

1. **Identical corpus.** All systems receive the same `chunks.text` ingested via each system's native API. No system receives an "optimized" version of the corpus.
2. **Identical eval set.** Same queries, same gold sets, same random seed (`42` for LongMemEval shuffle).
3. **Native defaults per system.** Each competitor runs with its publicly documented default configuration. Competitors are not adversarially tuned to lose; if the default config is what is published, it is what is evaluated.
4. **K cutoff fixed at 10.** Some systems default to 5 or 20; all are forced to `k=10` for comparability.
5. **Native embeddings provider per system.** nox-mem uses Gemini 3072d; each competitor uses its default provider. The `all-Gemini` controlled variant that equalizes this provider — originally planned as an optional side experiment — was **executed (rc4)** and is reported in §6.3.2: with both systems on Gemini 3072d over the full n=2,482 set, nox-mem outperforms Mem0 on both datasets and all five represented categories (whereas as-configured, §6.3, Mem0 wins LoCoMo), with four residual confounds declared and the task-type asymmetry ablated away (a generic-embedding re-run still wins by at least +0.05 nDCG@10 on both datasets).
6. **Uniform hardware.** Same VPS (Hostinger 8 cores / 16 GB RAM), localhost between systems except for external embeddings API calls.

Each adapter passes a pre-run smoke test:

```
result = adapter.search("test query", k=5)
assert len(result) >= 1
assert all('id' in r and 'score' in r for r in result)
```

Any adapter that fails the smoke test is documented as a gap (`[FAILED: <reason>]`) rather than omitted — consistent with §6.6.

6.6 Anti-cherry-pick statement

To prevent retroactive selection bias:

- **All 6 categories reported.** None is omitted because the result is unfavorable.
- **Both datasets reported.** LongMemEval n=100 + LoCoMo full, side by side. We do not cherry-pick whichever benefits nox-mem.
- **Latency — this principle is declared UNMET for the cross-system comparison, and the gap is stated rather than the principle quietly dropped.** Per-system latency percentiles were **not** captured uniformly in the 2026-06-15 run (§6.3), so this section reports **no** cross-system latency at all — not a favourable subset of it. nox-mem's own paths are reported separately in §5.7 with p50 **and p95** (KG path 2.9 / 5.7 ms; hybrid 653 / 706 ms); **p99 is not reported for every path**, and the opt-in cross-encoder row carries p50 only. An earlier version of this bullet claimed "p50 + p95 + p99 explicit", which was contradicted by §6.3 in this same section and was not true of §5.7 either.
- **Per-system per-category transparency.** The §6.4 table exposes every combination; there is no aggregate row that masks a pattern.
- **Gaps documented.** Systems that fail setup receive an explicit note; the comparison runs without the missing system, and the gap is recorded explicitly.
- **Explicit per-dataset breakdown (2026-05-23, rev3).** The Gemini hybrid@500 run revealed that the aggregate (0.0918) masks a decisive per-dataset result. We report all three rows explicitly:

System	nDCG@10 (aggregate)	nDCG@10 (LoCoMo-only)	Corpus	Mode
nox-mem FTS5@500	0.0466	—	500 (cap)	FTS5-only
nox-mem Gemini hybrid@500	0.0918	**0.1835**	500 (cap)	FTS5 + Gemini + RRF
mem0@500	**0.1315**	0.1315	500 (cap)	LLM rewrite + embed

LoCoMo-only result: nox-mem Gemini hybrid@500 = 0.1835 **exceeds** mem0@500 = 0.1315 by **+40%** on the conversational memory dimension. The aggregate (0.0918) falls below mem0 due to a **corpus-ordering artifact**: at the 500-chunk cap, the 5,882 LoCoMo chunks exhaust the cap before any LongMemEval chunk is ingested — the 10 LongMemEval queries have zero coverage, zeroing the nDCG for that dataset and pulling the aggregate down. Hybrid stack lift over FTS5@500: **+97%** (0.0466 →

0.0918), validating the architectural value of the stack even on a sparse corpus.

H2 finding (retained): FTS5-only@500 = 0.0466 vs mem0@500 = 0.1315 is **real and architectural** for FTS5-only mode — mem0's LLM-rewriting produces semantic generalization that isolated FTS5 cannot match. The full Gemini hybrid reverses this result on the conversational scope.

Mandatory disclosure: the aggregate ±0.05 falls within the inconclusive interval for n=20. The definitive arbiter is the canonical full-corpus run (uniform corpus without cap, full LoCoMo + LongMemEval for all systems). **Phase 2 gate uses BOTH** per-dataset + aggregate from the canonical run — not only the number that favors nox-mem.

6.7 Pre-registration

This section's methodology is locked in `specs/2026-05-23-Q4-comparison-execution-plan.md` prior to the 2026-05-24 run. The **2026-05-24 preliminary smoke** populated the first row of §6.3 (nox-mem combined: nDCG@10=0.6380, p50=8 ms, gold-hit 13/20 on 20 dry-run-sample queries) and validated that the retrieval pipeline works end-to-end on an eval-isolated DB. The **partial cross-system smoke of 2026-05-24** added the mem0 row (n=20, 500-chunk corpus cap): nDCG@10=0.8569, p50=273 ms, gold-hit 3/20 (15%) — with an explicit interpretation of the coverage vs. concentration trade-off in §6.3. The **canonical run was executed on 2026-06-15** on a dedicated pod (n=100/dataset, k=10, same-namespace fair), resolving the 2026-05-24 infrastructure abort (incident D76 — sustained CPU-steal on the shared host, see §7.1 L5). It produced real numbers for 3/6 systems (nox-mem, Mem0, agentmemory) and three documented gaps (§6.3.1), updating the §6.3 cells. The pre-registered `all-Gemini` controlled variant (§6.5, principle 5) was subsequently executed as **rc4** (§6.3.2) — equalizing the embedding provider over the full n=2,482 set — and supplied the §6.4 per-category breakdown; this was a planned confound-control experiment, not a post-hoc methodology change, and its residual confounds are declared in §6.3.2 per §6.6, with the task-type asymmetry tested by a dedicated ablation and shown not to drive the result. No methodology from §6.5–§6.6 was altered post-registration; the same-namespace re-query is a fair-comparison refinement (confound removal, §6.3) consistent with §6.5, not a change to corpus, eval set, or gold. Principles (§6.5), anti-cherry-pick (§6.6), and the general structure of this section are immutable post-run. Any methodological adjustment identified during execution is documented as an explicit follow-up rather than retroactively applied here.

Decision D43 defines the approval gate: nox-mem in top-3 on >=2 of the 4 key metrics (nDCG@10, R@10, MRR, latency). If not met, the 2026-05-25 session produces a remediation plan (pre-launch adjustments) rather than a direct launch.

6.8 Operational cost

6.8 Autonomy quantified — operational cost per memory system

The Q4 quality comparison (§6.3 – §6.6) reports retrieval *quality* under matched corpora. Operational *cost* — services, RAM, cold start, mandatory third-party credentials, setup commands — is the second axis on which a memory system can be evaluated, and is the axis where the nox-mem Autonomy pillar [83] is most legible. Table 2 summarizes the steady-state idle footprint of each system in its default self-host configuration.

Table 2 — Autonomy quantified: services, RAM, cold start, mandatory keys, setup commands. Headline: **~12× less RSS than EverOS, single process, no service stack.** Competitor numbers are *estimates* derived from each project's docker-compose defaults and documented system requirements (sources cited in the row). The nox-mem row is [**measured 2026-05-24, prod VPS, 6830 chunks live, uptime 9h28min**] via `ps -eo pid,rss,vsz,comm,args` against the production `nox-mem-api` process (single production process). See footnote [84] for full methodology including the cgroup `MemoryCurrent` vs process RSS distinction.

System	Services	RAM idle	Cold start	Mandatory third-party keys	Setup commands	Sources
nox-mem	**1** (SQLite file + Node process)	**~341 MB RSS** [measured 2026-05-24]	**<1 s**	**0** (offline-OK; embeddings optional)	**1** (`npm i && nox-mem reindex`)	This work; [85]

[83] Q/A/P strategic pivot of 2026-05-17 — three pillars (**Q**uality, **A**utonomy, **P**roduct).

[84] The `~341 MB RSS` figure for nox-mem in Table 2 is **measured 2026-05-24** on the production VPS (single production process, uptime 9h28min, 6830 chunks live, 100% vector coverage). Main process RSS = 349,276 KB via `ps -eo pid,rss,vsz,comm,args | grep dist/api-server.js`. The cgroup `MemoryCurrent` reported by `systemctl show nox-mem-api -p MemoryCurrent` is 727,064,576 bytes (~727 MB); the ~386 MB delta vs process RSS is SQLite memory-mapped I/O (chunks table, FTS5 index, vec0 index) — kernel-managed page cache, reclaimable on memory pressure, not exclusive process memory. Standard `ps`/`top` RSS is the canonical comparison metric used in Table 2 across all competitors. Original revision marked this as `~50 MB [estimated]` based on Node baseline + better-sqlite3 cache projections; the production measurement (initially denied during the first revision and granted later) replaced the estimate.

[85] The `~341 MB RSS` figure for nox-mem in Table 2 is **measured 2026-05-24** on the production VPS (single production process, uptime 9h28min, 6830 chunks live, 100% vector coverage). Main process RSS = 349,276 KB via `ps -eo pid,rss,vsz,comm,args | grep dist/api-server.js`. The cgroup `MemoryCurrent` reported by `systemctl show nox-mem-api -p MemoryCurrent` is 727,064,576 bytes (~727 MB); the ~386 MB delta vs process RSS is SQLite memory-mapped I/O (chunks table, FTS5 index, vec0 index) — kernel-managed page cache, reclaimable on memory pressure, not exclusive process memory. Standard `ps`/`top` RSS is the canonical comparison metric used in Table 2 across all competitors. Original revision marked this as `~50 MB [estimated]` based on Node baseline + better-sqlite3 cache projections; the production measurement (initially denied during the first revision and granted later) replaced the estimate.

System	Services	RAM idle	Cold start	Mandatory third-party keys	Setup commands	Sources
mem0	2 (Postgres + Qdrant)	~800 MB	~15 s	1 (OpenAI for embeddings)	~5	mem0 docker-compose defaults [86]
Letta	3 (Letta server + Postgres + OpenAI)	~1.5 GB	~30 s	1 (OpenAI)	~8	Letta self-host guide [87]
Zep OSS	2 (Zep + Postgres; 3 with the local embedder)	~1.2 GB	~30 s	**1 mandatory** (a paid LLM key — OpenAI *or* Anthropic; the server aborts at startup without it)	~6	Zep v0.27.2 source [88]
EverOS / EverMind-AI	**5** (MongoDB + Elasticsearch + Milvus + Redis + Postgres)	**~4 GB+**	**~60 s**	2–3 (LLM + embedding + optional reranker)	~15+	EverMind-AI docker-compose [89]

[86]mem0 default self-host requires Postgres + Qdrant + an OpenAI key for embeddings (or a configured alternative provider). Counts: 2 services + 1 mandatory third-party key. Source: mem0 README and `docker-compose.yml` defaults at github.com/mem0ai/mem0.

[87]Letta default self-host requires the Letta server, Postgres, and an OpenAI key (or alternative LLM provider) for the agent loop. Counts: 3 components + 1 mandatory third-party key. Source: Letta self-host documentation at docs.letta.com and github.com/letta-ai/letta.

[88]Zep OSS requires the Zep service container + Postgres, plus a third container (the local embedder) if embeddings are not routed to a paid vendor. **One paid LLM key is mandatory in every configuration:** `pkg/llms/llm_base.go` constructs an LLM client unconditionally — including for the empty service string, which falls through to OpenAI — and both back-ends `log.Fatal` on an empty key (`llm_openai.go`: `ZEP_OPENAI_API_KEY is not set`; `llm_anthropic.go`: `ZEP_ANTHROPIC_API_KEY is not set`). The embedder, by contrast, is not vendor-locked: `pkg/llms/embeddings.go` routes `Service: local` to `pkg/llms/embeddings_local.go`, which POSTs to the local NLP service, and `ZepAnthropicLLM.EmbedTexts` returns "not implemented. use a local embedding model". Counts: 2 services + 1 mandatory paid LLM key. Source: `getzep/zep` at tag `v0.27.2` — the image we ran — read 2026-09-10.

[89]EverMind-AI / EverOS docker-compose declares MongoDB + Elasticsearch + Milvus + Redis + Postgres = 5 services, plus 2–3 third-party API keys for LLM, embedding, and (optional) reranker. Counts confirmed against the published `docker-compose.yml` in the EverMind-AI repo **as of 2026-06-15**. **Caveat.** Re-probed 2026-09-10: that file returns HTTP 404 at the repository root and the project now ships as a pip-installable local-first library (v1.3.1) — the count was correct when taken and has since expired; it is retained because the operational-footprint comparison it supports was made against that version, and silently updating it would misdate the comparison. The ~4 GB RAM-idle figure is the sum of documented minimum requirements for each service's container.

System	Services	RAM idle	Cold start	Mandatory third-party keys	Setup commands	Sources
LightRAG	2 (Neo4j + vector DB)	~1 GB	~20 s	1 (LLM provider for KG extraction)	~6	LightRAG repo defaults [90]

Reading the table. Three rows of the cost matrix translate directly into Autonomy:

1. **Services column.** Every additional service is an additional failure mode, an additional security-patching surface, and an additional vendor that must be available on the day a user spins up the system. nox-mem ships as a single Node process operating on a single SQLite file; the only durable on-disk artifact is `nox-mem.db`. mem0/Zep/Letta/LightRAG each require >=1 database container and at least one external LLM/embedding provider. EverOS requires five containers, three of which are heavyweight infrastructure (MongoDB, Elasticsearch, Milvus). The single-service property is what makes "open `nox-mem.db` in `sqlite3` and inspect everything" a literal operation, not a euphemism.

2. **Mandatory third-party keys column.** A system that requires an OpenAI key by default is not autonomous regardless of license — the user is dependent on one specific vendor's pricing, rate limits, and terms of service. nox-mem treats embeddings as optional (FTS5-only retrieval is a valid degraded mode; §4) and is provider-agnostic when embeddings are enabled (Gemini default, Ollama-local feasible — §7.1 L2). Zep requires a paid LLM key of one of two vendors (OpenAI by default, Anthropic selectable) and refuses to start without it; what is *not* vendor-locked in Zep is the embedder, which can run keyless against its own local embedding service at the cost of a third container ([91]). Letta documents OpenAI as its default provider.

3. **Cold start column.** A `<1s` cold start is what makes self-host *try-before-deciding* — the user can `npm i`, run one command, see results, and decide.

[90] LightRAG defaults to Neo4j for the knowledge graph + a vector store (Qdrant/Milvus/etc.), plus one LLM provider key for entity/relation extraction during indexing. Counts: 2 services + 1 mandatory third-party key. Source: LightRAG README at github.com/HKUDS/LightRAG.

[91] Zep OSS requires the Zep service container + Postgres, plus a third container (the local embedder) if embeddings are not routed to a paid vendor. **One paid LLM key is mandatory in every configuration:** `pkg/llms/llm_base.go` constructs an LLM client unconditionally — including for the empty service string, which falls through to OpenAI — and both back-ends `log.Fatal` on an empty key (`llm_openai.go`: `ZEP_OPENAI_API_KEY is not set`; `llm_anthropic.go`: `ZEP_ANTHROPIC_API_KEY is not set`). The embedder, by contrast, is not vendor-locked: `pkg/llms/embeddings.go` routes `Service: local` to `pkg/llms/embeddings_local.go`, which POSTs to the local NLP service, and `ZepAnthropicLLM.EmbedTexts` returns "not implemented. use a local embedding model". Counts: 2 services + 1 mandatory paid LLM key. Source: `getzep/zep` at tag `v0.27.2` — the image we ran — read 2026-09-10.

A ~60s cold start with five containers is what makes EverOS effectively a "build a small team to evaluate" decision, not an individual decision.

Caveat — RAM measurement methodology. The competitor RAM figures are *idle* (i.e., process started, no queries served, no ingestion in progress) and are *estimates* read from each project's documented system requirements and `docker stats` defaults in the published docker-compose files. They are not from a head-to-head benchmark on a single host. A side-by-side measurement on a controlled 4-vCPU / 8-GB host is a §7.2 future-work item (F-cost-bench). The nox-mem ~341 MB RSS figure is **measured** on the production VPS via `ps -eo pid,rss,vsz,comm,args` (single production process, uptime 9h28min, 6830 chunks live); see footnote [92] for full methodology including the cgroup `MemoryCurrent` vs process RSS distinction.

6.9 Operational reproducibility as evidence

The §6.3 quality split and the §6.8 cost matrix report *outcomes*. A third, harder-to-fake signal emerged from the **act of running the benchmark itself**: the operational effort required to get each system to produce a number is a measurement of its dependency surface.

On the 2026-06-15 pod, nox-mem ingested the full offered corpus — **6,830 documents carrying 6,822 distinct ids**, of which its eval loader retained the 6,822 (§6.3.2) — into a single SQLite file in one process with **zero incidents**. The competitors did not fare as smoothly:

- **Mem0** exhausted the pod's process-ID limit three times — its default telemetry (PostHog) leaks one thread per operation, and at benchmark scale this wedged the host until ingest and search were split into separate processes, telemetry was disabled (`MEM0_TELEMETRY=False`), and the vector backend was swapped from Chroma to faiss. Its LoCoMo store still lost ~5% of vectors to the leak before the workaround stabilized.
- **Zep** did not run *on this pod*: Docker is impossible on an unprivileged RunPod kernel (§6.3.1). That is a property of the environment, not of Zep — it ran on hosts with a working Docker daemon, both in the superseded 2026-05-25 smoke (artifact in `eval/q4-comparison/output/zep.json`) and again on 2026-09-10. What constrains Zep on the autonomy axis is not Docker but the key (§6.8).

[92] The ~341 MB RSS figure for nox-mem in Table 2 is **measured 2026-05-24** on the production VPS (single production process, uptime 9h28min, 6830 chunks live, 100% vector coverage). Main process RSS = 349,276 KB via `ps -eo pid,rss,vsz,comm,args | grep dist/api-server.js`. The cgroup `MemoryCurrent` reported by `systemctl show nox-mem-api -p MemoryCurrent` is 727,064,576 bytes (~727 MB); the ~386 MB delta vs process RSS is SQLite memory-mapped I/O (chunks table, FTS5 index, vec0 index) — kernel-managed page cache, reclaimable on memory pressure, not exclusive process memory. Standard `ps`/`top` RSS is the canonical comparison metric used in Table 2 across all competitors. Original revision marked this as ~50 MB [estimated] based on Node baseline + better-sqlite3 cache projections; the production measurement (initially denied during the first revision and granted later) replaced the estimate.

- **Letta** ran but at ~16 min/query, making a 100-query benchmark a multi-day proposition.
- **agentmemory** silently persisted store state across sessions, contaminating a LoCoMo run with leftover LongMemEval vectors until the store was reset.

Scope of this subsection (2026-09-10). The non-runs above are non-runs *of the 2026-06-15 canonical pod*, and the list is shrinking as upstream barriers lift. EverOS left it first (§6.3.3), taking the count of non-running competitors from three to two; **Zep left it on the same day** (§6.3.4), brought up on a host with a working Docker daemon and a paid key and swept over the full n=2,482 set, which takes the count to **one — Letta**. The operational finding that survives either departure is the one verified in source rather than inferred from our environment: Zep cannot boot without a paid LLM key (§6.8).

The gaps are not neutral omissions — they are a deployability penalty. Standard benchmarking convention records a non-running system as a blank cell and moves on. We make a stronger, but carefully bounded, claim: *the dimension on which this one system could not produce a number is precisely the dimension nox-mem is engineered to win.* Each failure maps to a concrete operational deficit that nox-mem does not carry:

System	What constrains it (measured / source-verified)	Operational axis it fails	nox-mem on the same axis
Zep	one paid LLM key (OpenAI or Anthropic) is mandatory in every configuration; the server calls `log.Fatal` at startup without it — verified in the v0.27.2 source ([93])	dependency footprint / autonomy	1 process, **0 mandatory keys**, FTS5-only is a valid degraded mode
Letta	ran, but retrieval routes through a full LLM agent turn at ~16 min/query	query efficiency	2.9 ms KG path / 653 ms hybrid — **5–6 orders of magnitude faster**

[93] Zep OSS requires the Zep service container + Postgres, plus a third container (the local embedder) if embeddings are not routed to a paid vendor. **One paid LLM key is mandatory in every configuration:** `pkg/llms/llm_base.go` constructs an LLM client unconditionally — including for the empty service string, which falls through to OpenAI — and both back-ends `log.Fatal` on an empty key (`llm_openai.go`: `ZEP_OPENAI_API_KEY is not set`; `llm_anthropic.go`: `ZEP_ANTHROPIC_API_KEY is not set`). The embedder, by contrast, is not vendor-locked: `pkg/llms/embeddings.go` routes `Service: local` to `pkg/llms/embeddings_local.go`, which POSTs to the local NLP service, and `ZepAnthropicLLM.EmbedTexts` returns "not implemented. use a local embedding model". Counts: 2 services + 1 mandatory paid LLM key. Source: `getzep/zep` at tag `v0.27.2` — the image we ran — read 2026-09-10.

System	What constrains it (measured / source-verified)	Operational axis it fails	nox-mem on the same axis
EverMind-AI *(as of 2026-06-15; see note)*	needed 5 services + 2 mandatory paid third-party keys (OpenRouter + DeepInfra)	dependency footprint / autonomy	1 process, **0 mandatory keys**, provider-agnostic

> **The EverMind-AI row is dated (2026-06-15) and its constraint has since lifted.** That version needed five services and two mandatory paid keys, and the operational comparison above was made against it. The current version installs as a local-first library, ran on 2026-09-10, and **produced a quality number that beats nox-mem** (§6.3.3). The row is kept, dated, rather than deleted: silently removing it would misdate a comparison that was accurate when taken. The deployability claim below is therefore about Letta only — Zep also produced a number on 2026-09-10 (§6.3.4), and its own row below stays dated for the same reason: the paid-key finding was verified in the v0.27.2 source and is unaffected by the run.

Honest bound (per §6.6). We do *not* infer that nox-mem *retrieves better* than this one system — we hold no comparable quality number for it (Letta runs, but at ~16 min/query it was never swept — §6.3.1), and in a configuration that gives it the time its agent turn costs it may retrieve competitively. The claim is narrower, and stronger for being narrow: on the **deployability / efficiency / autonomy axis** — the axis that decides whether an individual developer, or an embedded agent that must live in its own process on commodity hardware, can use the system *at all* — **one of five competitors could not be made to produce a single result**, while nox-mem produced one from one file with zero external services. For that user, a system you cannot run is not "untested": it is unusable, and unusable is the worst score on the operational axis. A lighter, dependency-free design is not a footnote to the quality comparison — it is the reason nox-mem has a number in every cell of §6.3 and one competitor does not.

The reproducibility of nox-mem's run — one file, one process, one command, re-confirmed live at **415 MB RSS on the 70.7k-chunk production corpus** on 2026-06-15 — is the operational expression of the Autonomy pillar quantified in §6.8. **A memory system you can run from a single file you own is a different category of dependency than one that requires Docker, a vector database, a telemetry pipeline, and a third-party embedding key to return its first result.**

---

7. Limitations and Future Work

7.1 Limitations

L1 — Explicit-ingestion dependency (no zero-shot corpus coverage) nox-mem retrieves only what has been explicitly ingested via `ingestFile()`, `ingest-entity`, or the inotifywait watcher pipeline. There is no mechanism to answer queries over arbitrary external corpora at query time. This is a deliberate design constraint — the system is optimized for an agent's *own* accumulated memory, not general-purpose retrieval augmentation — but it means that coverage is bounded by ingestion discipline. A corpus that has never been ingested produces zero recall regardless of query quality. Users bootstrapping the system must explicitly run `nox-mem reindex` over existing files before the hybrid search layer is useful. See §3.1 (ingestion pipeline).

L2 — Gemini API dependency for embeddings (cost + outbound network) The semantic retrieval layer (Layer 2) depends on Google's `gemini-embedding-001` model (3072 dimensions). This introduces two constraints: (a) every vectorization call requires outbound network access and a valid `GEMINI_API_KEY`, meaning an air-gapped deployment falls back to FTS5-only retrieval with no semantic recall; (b) API cost scales with corpus size — at the 2026-05-24 corpus of ~69k chunks, a full re-vectorization pass takes approximately 30–40 minutes at quota limits of the free tier. The Autonomy pillar of the Q/A/P strategy explicitly calls out "provider your choice, zero vendor lock-in" as a long-term goal; local embedding substitution (e.g., `nomic-embed-text` via Ollama) is architecturally feasible but not validated against the canonical eval set. bring-your-own-key (BYOK) partial autonomy is available: users who supply their own Gemini API key operate without per-query billing exposure in the default free tier.

L3 — Single-instance architecture (no distributed sharding or replication) The system runs on a single SQLite file per agent database. WAL mode provides concurrent read safety, but there is no horizontal sharding, no replication across nodes, and no distributed coordination layer. The current production corpus (79,220 chunks summed across 7 databases — 67,724 of them in the main store — on a 4-vCPU / 8GB KVM4, as of 2026-09-09) operates comfortably within these bounds, but the architecture does not generalize to multi-tenant deployments or corpora significantly exceeding the single-node memory/storage envelope. Distributed SQLite extensions (e.g., `cr-sqlite` CRDT-based replication) exist but are explicitly out of scope for v1. This is a known architectural decision, not an oversight.

L4 — No write-side concurrency control (last-writer-wins) Chunk ingestion operates under an optimistic concurrency model: `ingestFile()` deletes existing chunks for the source file and re-inserts in a single transaction, but there is no row-level locking or version fence against concurrent

ingest of the same source file from two processes. In practice the inotify-wait watcher and manual CLI calls rarely overlap, and the WAL journal prevents data corruption; however, two concurrent ingest calls on the same file produce non-deterministic chunk counts. The `withOpAudit()` wrapper (`staged/1.7a/edits/op-audit.ts`) does not add a mutual-exclusion layer for ingest — it targets destructive bulk operations (reindex, consolidate, crystallize). Production mitigations are operational (systemd service prevents concurrent watcher processes; cron stagger of 5 minutes between agents), not architectural.

L5 — Evaluation sample size and canonical run gap (resolved 2026-06-15 / 2026-06-29) The 2026-05-24 cross-system run was a 20-query methodology smoke over an eval-isolated DB (5,882 LoCoMo + 940 LongMemEval chunks), not the canonical run; the first canonical attempt was **aborted** (incident D76: sustained CPU-steal on the shared production host, 51-97% over 48h, batch 005 0/50 in 23h). The canonical run was subsequently executed on dedicated infrastructure on **2026-06-15** (n=100/dataset, 3/6 systems producing numbers + 3 documented gaps, §6.3), and the **controlled-embedding variant (rc4)** was run on **2026-06-29** at full scale (n=2,482, both systems on Gemini 3072d, §6.3.2 / §6.4). The §6 competitive figures are therefore **settled, not `[deferred]`**; the residual limitation is the confounds declared on rc4 in §6.3.2, not sample size, and the one asymmetry that favored nox-mem (embedding task type) was ablated and ruled out. The earlier nox-mem smoke figure (nDCG@10 = 0.6380 combined) is not directly comparable to the G5 V3 entity-eval figure (0.6237) because the eval corpus and query set differ.

L6 — Cross-system comparison is methodologically partial One of five competitors could not be evaluated at all. After the canonical run (2026-06-15, n=100/dataset) and the rc4 embedding-matched run (2026-06-29, full n=2,482), Mem0 and agentmemory produced real numbers alongside nox-mem; the **500-chunk cap was a property of the superseded 2026-05-24 smoke only** and no longer applies. The standing limitation is **deployability coverage**: **Letta** (agent-OS, ~16 min/query) produced no number in the unprivileged single-node environment (§6.3.1) — a deployability gap on the operational axis, not a retrieval-quality claim against it. Two systems that were gaps of the canonical run have since closed: **EverMind-AI / EverOS** and **Zep**, both swept over the full n=2,482 set on 2026-09-10 once the barriers of 2026-06-15 (third-party credentials; a host with a real Docker daemon) were lifted — reported in §6.3.3. The rc4 head-to-head additionally carries the four residual confounds declared in §6.3.2 (its fourth potential confound, embedding task-type asymmetry, was ablated and ruled out). See §6.6 (anti-cherry-pick statement).

L7 — Latency comparison conflates transport classes Cross-system latency was not captured uniformly. The canonical (2026-06-15) and rc4 runs did not record per-system latency percentiles under a normalized transport, and the

only side-by-side figures — from the superseded 2026-05-24 smoke — compared nox-mem localhost retrieval against a Docker-in-Docker HTTP call, i.e. different transport classes, not a head-to-head speed claim. nox-mem's operational latency is therefore reported standalone in §5.7 (KG path p50 2.9 ms, hybrid p50 653 ms, re-measured 2026-06-15), not as a cross-system comparison. A normalized-transport latency benchmark (all systems behind one gateway) is future work (§7.2).

L8 — Pain signal is directional but not statistically significant in isolation The "pain-weighted hybrid memory" framing rests primarily on the additive salience formula (§5.1.2) and section-aware ranking (§5.1.3). The `pain` dimension contributes W_PAIN = 0.10 of the salience weight, but its isolated causal contribution has not been validated to statistical significance: the E10 pain ablation (`paper/publication/paper-draft-sec4-7.md` §5.5) reports Δ = +0.0065 with 95% CI [−0.0143, +0.0338] on n = 31, directional but not significant. The current production corpus has 91.74% of chunks at the default `pain = 0.2` (as of 2026-06-04), providing insufficient variance for a precise estimate. A definitive pain signal ablation requires a corpus where pain scores span the full [0.1, 1.0] range. See §5.7.

7.2 Future Work

F1 — A2 Tier 3 P5: production-ready encrypted memory (in flight) Phase 5 of the A2 Tier 3 roadmap targets a full SQLCipher-encrypted memory store with Ed25519-signed audit checkpoints (deployed). The signed checkpoint chain enables tamper-evident audit across destructive operations (`reindex`, `consolidate`, `crystallize`) without requiring a central trust authority. P5 closes the Tier 3 arc by integrating encryption key management with the existing `withOpAudit()` wrapper and the Tier 3 reads-audit layer. Target deployment: post-GTM Phase 2 launch, estimated 2026-05-25.

F2 — F10 Phase C/D: shadow tracker empirical A/B for ranking changes F10 Phase A (`/observability/health.html`) and Phase B (`/observability/evals.html`) are deployed (§5.7.4, decision D53). Phase C targets a shadow-mode query logger that captures production queries, executes them against a candidate ranking config in parallel, and accumulates query-level nDCG deltas before any promotion decision. Phase D operationalizes this into a pre-promotion gate: any ranking change (boost weight adjustment, mutex threshold, salience weight) that has not accumulated >=50 shadow queries with $p < 0.05$ improvement is blocked from reaching the production endpoint. This closes the observability gap — currently the shadow mode is a flag toggle, not an integrated eval pipeline. #### F3 — Per-method benchmark Phase B: cross-method nDCG optimization

The Q4 per-method benchmark (§6) establishes the cross-system baseline. Phase B targets per-query-type boost calibration: given that keyword queries respond differently from natural-language queries (G10c §5.1.4), and single-hop vs multi-hop have opposing mutex trade-offs (G10b §5.1.4), a routing layer that selects ranking parameters based on query-type classification has measurable potential upside. Estimated Lab Q1 item.

F4 — EverMemBench equivalence: honest comparison against EverMind-AI dataset EverMind-AI publishes standardized results on EverMemBench (EverCore 83% LongMemEval / 93% LoCoMo; HyperMem 92.73% LongMemEval). Running nox-mem on EverMemBench with the same evaluation protocol closes the benchmark gap and provides a reviewer-grade comparison for this submission. This is gated on EverMind-AI repository availability (currently unavailable) or an alternative comparable dataset. Estimated Lab Q1 priority if the repository returns.

F5 — Neural reranker: cross-encoder rerank post-RRF The current retrieval stack terminates at RRF fusion (§4.1). A cross-encoder reranker — receiving the top-K RRF candidates and the original query as a pair — is the standard next step in multi-stage retrieval and typically yields +3–8% nDCG@10 over bi-encoder baselines (see e.g., Nogueira & Cho[94] on MS MARCO; the current open-weight candidate is the Qwen3 reranker[95]). The Autonomy constraint favors a locally-runnable cross-encoder (e.g., `cross-encoder/ms-marco-MiniLM-L-6-v2` via sentence-transformers, ~66MB) over a cloud inference call, keeping the retrieval stack fully offline-capable. Estimated Lab Q1/Q2.

F6 — Lab Q1 scale validation: 250k chunk corpus The main store held 67,724 chunks with complete vector coverage as of 2026-09-09, which is below the ~100k-vector threshold at which exact-search latency begins to degrade. An earlier internal record placed the main store near 95k as of 2026-06-04; that figure is not re-verifiable from the current corpus, and the net direction of change since has been downward (retention-based pruning), so the threshold is further away than the June figure implied. The Lab Q1 roadmap targets a 250k chunk corpus to validate: (a) sqlite-vec ANN recall at scale (current exact-search; approximate search becomes necessary past ~100k vectors at reasonable latency

[94]Nogueira & Cho, *Passage Re-ranking with BERT*, 2019. arXiv:1901.04085. The work that established the cross-encoder as a re-ranking stage over a first-stage candidate pool: the MS MARCO result cited in §7.2 F5, and the stage whose **presence or absence** is the confound of §6.3.3–§6.3.4 — EverOS requires one, nox-mem fuses by RRF[⁀rrf] without one, Zep has neither. Our own measurement of the stage (§5.1.7, −0.96 pp) used a 22 M distillation[⁀minilm], ~180× smaller than the 4 B model EverOS runs[⁀qwen3embed], so it is not carried over as a bound.

[95]Zhang, Li, Long, Zhang, Xie *et al.*, *Qwen3 Embedding: Advancing Text Embedding and Reranking Through Foundation Models*, 2025. arXiv:2506.05176. Named in §7.2 F5 as the current open-weight reranker candidate **for nox-mem's own stack, where it is not run**. It is not, however, absent from this study: EverOS reranks internally with a Qwen3 reranker, and §6.3.1 records the `rerank_n = 50` ceiling that this imposes on the candidate set measured there. The model appears here as a component of a *compared system*, never of ours.

targets); (b) salience formula stability (the recency component decays over a longer history window); (c) FTS5 BM25 IDF calibration (with more documents, rare-term IDF weights shift). This scale-validation item has no committed spec yet; it is gated on `NOX_SALIENCE_MODE=active` remaining stable through the GTM Phase 2 feedback cycle.

F7 — Multilingual corpus coverage: Portuguese and Spanish The current evaluation corpus is English-dominant (LongMemEval and LoCoMo are English datasets; the internal entity-eval golden set mixes English and Portuguese). The FTS5 `unicode61` tokenizer with porter stemmer does not stem Portuguese or Spanish tokens correctly (e.g., "decisão" stems to "decisa" rather than "decid-"; Spanish gerunds lose morphological overlap). The Gemini semantic layer partially compensates via cross-lingual embedding space, but there is no explicit multilingual evaluation. A Portuguese golden set is a natural next step given the production operational language of the corpus; **Quati** (Bueno et al., 2024, arXiv:2404.06976), a Brazilian-Portuguese IR dataset annotated by native speakers, is a strong candidate benchmark for that evaluation. This is deferred to post-launch community feedback intake.

F8 — GTM Phase 2 launch and community feedback intake The Q/A/P roadmap defines GTM Phase 2 as gated on D43 (nox-mem top-3 in at least 2 of 4 key metrics — nDCG@10, R@10, MRR, latency). With the canonical and rc4 runs complete, D43 is satisfied; the public OSS release is the remaining gate-dependent step. Post-launch, community feedback from the OSS release is expected to surface real-world limitation patterns not visible in the synthetic golden sets (e.g., corpora with high image-to-text OCR content, multi-language mixes, or very short memory fragments < 20 words that the current chunker merges). The feedback cycle directly informs Lab Q2 priorities.

---

8. Conclusion

nox-mem demonstrates that persistent, searchable, and shareable memory for AI agent fleets is achievable with commodity infrastructure: a single VPS, one SQLite file, and a provider-agnostic embedding layer — Gemini in every configuration measured here, with FTS5-only retrieval as a valid keyless degraded mode (§4). The hybrid search system consistently outperforms single-method retrieval across the evaluated corpora (§5.1, §6.3). Multilingual behaviour is **not** among the claims: the evaluation corpus is English-dominant and Portuguese/Spanish coverage is declared future work (§7.2 F7). The LLM-powered knowledge graph extracts typed entities and relations that the earlier regex path did not, and temporal decay keeps the graph current without manual curation (§5.6). No head-to-head extraction-yield ratio against the regex path is reported here — an earlier draft carried a 15× figure that the evaluation does not support. The

Wave A empirical evaluation (§5) established nDCG@10 = 0.6237 on the entity-flavored golden set (+78.8% relative over the G3 baseline), with `section_boost` identified as the dominant driver (99.85% of the lift recovered by A3 alone) and the additive salience formula validated by the `active > shadow` reversal. The G10d conditional mutex evolution (§5.1.4, deployed 2026-05-21) consolidates the canonical boost stack `section_boost × source_type_boost (Hard Mutex gated by query_entity_count <= 2) × salience v2 additive` in production, recovering multi-hop and adversarial regressions with contained dilution on single-hop. The F10 layer (§5.7.4, decision D53) makes production state verifiable at any time via the Phase A dashboard (`/observability/health.html`) + Phase B dashboard (`/observability/evals.html`).

The §6 Q4 COMPARISON is pre-registered (`specs/2026-05-23-Q4-comparison-execution-plan.md`). After a methodology smoke (2026-05-24, n=20) and an infrastructure abort (incident D76, §7.1 L5), the **canonical run executed 2026-06-15** (n=100/dataset, 6 systems: 3 produced numbers, 3 documented gaps — §6.3), and the **all-Gemini controlled variant (rc4) executed 2026-06-29** (full n=2,482 — §6.3.2 / §6.4). Under each system's native embedder the two leaders split — Mem0 wins LoCoMo, nox-mem wins LongMemEval (§6.3); with the embedding provider equalized, nox-mem outperforms Mem0 on both datasets and all five represented categories (§6.3.2). Both readings are reported side by side per §6.6, with four residual confounds declared and the task-type asymmetry ablated away. The D43 gate (top-3 on at least 2 of the 4 key metrics) is satisfied, unlocking the GTM Phase 2 pre-launch defense.

The cross-agent intelligence layer transforms isolated agent memories into a collaborative knowledge base, enabling institutional learning across the fleet. Combined with the live dashboard, the system provides full observability into the collective memory of the agent organization.

Repository: github.com/totobusnello/memoria-nox

Appendices — moved out

The manuscript carries no appendix. The seven operational appendices of earlier revisions — **A.** Knowledge Graph v2, **B.** Cross-Agent Intelligence, **C.** MCP Server Interface, **D.** HTTP API Server, **E.** Operational Infrastructure, **F.** Dashboard Integration, **G.** Evolution History — are included in the anonymized supplementary material rather than in this PDF. They document how the deployment is wired rather than what was measured, so none of them is load-bearing for a claim made here. Each is **named here rather than silently dropped**: absence of something that used to be present has to be visible, or a reader who was looking for it cannot tell removal from oversight.

References and Footnotes

Related-systems references

Autonomy table (Table 2) sources

Academic references

Every entry below carries a resolvable identifier (arXiv ID or DOI). Full BibTeX in `paper/refs.bib`. arXiv IDs were verified against the arXiv API on 2026-09-09 — ID, first author and title checked to match, rather than transcribed from memory.

Internal references